

HW5 - Econ 144

Mason Lee

2025-11-19

Packages

```
library(readr)
library(lattice)
library(foreign)
library(MASS)
library(car)
```

```
## Loading required package: carData
```

```
require(stats)
require(stats4)
```

```
## Loading required package: stats4
```

```
library(KernSmooth)
```

```
## KernSmooth 2.23 loaded
## Copyright M. P. Wand 1997-2009
```

```
library(fastICA)
library(cluster)
library(leaps)
library(mgcv)
```

```
## Loading required package: nlme
```

```
## This is mgcv 1.9-1. For overview type 'help("mgcv-package")'.
```

```
library(rpart)
library(pan)
library(mgcv)
library(DAAG)
```

```
##
## Attaching package: 'DAAG'
```

```
## The following object is masked from 'package:car':  
##  
##      vif
```

```
## The following object is masked from 'package:MASS':  
##  
##      hills
```

```
library("TTR")  
library(tis)
```

```
##  
## Attaching package: 'tis'
```

```
## The following object is masked from 'package:TTR':  
##  
##      lags
```

```
## The following object is masked from 'package:mgcv':  
##  
##      ti
```

```
require("datasets")  
require(graphics)  
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':  
##      method      from  
##      as.zoo.data.frame zoo
```

```
##  
## Attaching package: 'forecast'
```

```
## The following object is masked from 'package:tis':  
##  
##      easter
```

```
## The following object is masked from 'package:nlme':  
##  
##      getResponse
```

```
require(astsa)
```

```
## Loading required package: astsa
```

```
##  
## Attaching package: 'astsa'
```

```
## The following object is masked from 'package:forecast':  
##  
##      gas
```

```
library(RColorBrewer)
library(plotrix)
library(tsibble)
```

```
## Registered S3 method overwritten by 'tsibble':
##   method          from
##   as_tibble.grouped_df dplyr
```

```
##
## Attaching package: 'tsibble'
```

```
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, union
```

```
library(ggplot2)
library(tidyr)
library(USgas)
library(fpp3)
```

```
## -- Attaching packages ----- fpp3 1.0.2 --
```

```
## v tibble      3.2.1      v tsibbledata 0.4.1
## v dplyr       1.1.4      v feasts      0.4.2
## v lubridate   1.9.4      v fable       0.4.1
```

```
## -- Conflicts ----- fpp3_conflicts --
```

```
## x feasts::ACF()          masks nlme::ACF()
## x dplyr::between()       masks tis::between()
## x dplyr::collapse()      masks nlme::collapse()
## x lubridate::date()      masks base::date()
## x lubridate::day()       masks tis::day()
## x dplyr::filter()        masks stats::filter()
## x lubridate::hms()        masks tis::hms()
## x tsibble::intersect()   masks base::intersect()
## x lubridate::interval()  masks tsibble::interval()
## x dplyr::lag()           masks stats::lag()
## x lubridate::month()     masks tis::month()
## x lubridate::period()    masks tis::period()
## x lubridate::POSIXct()   masks tis::POSIXct()
## x lubridate::quarter()   masks tis::quarter()
## x dplyr::recode()        masks car::recode()
## x dplyr::select()        masks MASS::select()
## x tsibble::setdiff()     masks base::setdiff()
## x lubridate::today()     masks tis::today()
## x tsibble::union()       masks base::union()
## x lubridate::year()      masks tis::year()
## x lubridate::ymd()       masks tis::ymd()
```

Textbook A: Forecasting for Economics and Business

Data

```
fm <- read.csv("fmhpi_master_file.csv")
head(fm)
```

```
##   Year Month GEO_Type GEO_Name GEO_Code Index_NSA Index_SA
## 1 1975     1   State      AK      .   35.05465 35.46056
## 2 1975     2   State      AK      .   35.55635 36.03117
## 3 1975     3   State      AK      .   36.07694 36.44777
## 4 1975     4   State      AK      .   36.62437 36.72499
## 5 1975     5   State      AK      .   37.22699 36.95556
## 6 1975     6   State      AK      .   37.88329 37.24204
```

```
table(fm$GEO_Type)
```

```
##
##   CBSA   State    US
## 235683 31059    609
```

```
msa <- subset(fm, GEO_Type == "CBSA")
MSA1_name <- "Oxnard-Thousand Oaks-Ventura CA"
MSA2_name <- "Los Angeles-Long Beach-Anaheim CA"
```

```
msa1 <- subset(msa, GEO_Name == MSA1_name)
msa2 <- subset(msa, GEO_Name == MSA2_name)
```

```
msa1$Date <- as.Date(paste(msa1$Year, msa1$Month, "01", sep = "-"))
msa2$Date <- as.Date(paste(msa2$Year, msa2$Month, "01", sep = "-"))
```

```
msa1 <- msa1[order(msa1$Date), ]
msa2 <- msa2[order(msa2$Date), ]
```

```
head(msa1)
```

```
##           Year Month GEO_Type           GEO_Name GEO_Code Index_NSA
## 182701 1975     1   CBSA Oxnard-Thousand Oaks-Ventura CA    37100 13.90367
## 182702 1975     2   CBSA Oxnard-Thousand Oaks-Ventura CA    37100 14.00809
## 182703 1975     3   CBSA Oxnard-Thousand Oaks-Ventura CA    37100 14.20650
## 182704 1975     4   CBSA Oxnard-Thousand Oaks-Ventura CA    37100 14.50231
## 182705 1975     5   CBSA Oxnard-Thousand Oaks-Ventura CA    37100 14.74632
## 182706 1975     6   CBSA Oxnard-Thousand Oaks-Ventura CA    37100 14.85763
##           Index_SA      Date
## 182701 13.89219 1975-01-01
## 182702 13.99511 1975-02-01
## 182703 14.14028 1975-03-01
## 182704 14.37169 1975-04-01
## 182705 14.59592 1975-05-01
## 182706 14.69720 1975-06-01
```

```
head(msa2)
```

```
##           Year Month GEO_Type          GEO_Name GEO_Code Index_NSA
## 140071 1975      1    CBSA Los Angeles-Long Beach-Anaheim CA    31080 15.72848
## 140072 1975      2    CBSA Los Angeles-Long Beach-Anaheim CA    31080 15.88764
## 140073 1975      3    CBSA Los Angeles-Long Beach-Anaheim CA    31080 16.15511
## 140074 1975      4    CBSA Los Angeles-Long Beach-Anaheim CA    31080 16.53437
## 140075 1975      5    CBSA Los Angeles-Long Beach-Anaheim CA    31080 16.85256
## 140076 1975      6    CBSA Los Angeles-Long Beach-Anaheim CA    31080 17.00757
##           Index_SA      Date
## 140071 15.70656 1975-01-01
## 140072 15.86993 1975-02-01
## 140073 16.08712 1975-03-01
## 140074 16.40525 1975-04-01
## 140075 16.71223 1975-05-01
## 140076 16.86140 1975-06-01
```

11.1

```
dat <- merge(
  data.frame(
    Date = msa1$Date,
    g1 = c(NA, 100 * diff(log(msa1$Index_SA))) # Oxnard growth
  ),
  data.frame(
    Date = msa2$Date,
    g2 = c(NA, 100 * diff(log(msa2$Index_SA))) # LA growth
  ),
  by = "Date"
)
dat <- dat[complete.cases(dat), ]

summary(dat)
```

```
##           Date           g1           g2
## Min.      :1975-02-01   Min.      : -9.64018   Min.      : -11.23512
## 1st Qu.:1987-09-23   1st Qu.:  0.01462   1st Qu.:  -0.04954
## Median :2000-05-16   Median :  0.55311   Median :   0.59246
## Mean    :2000-05-16   Mean    :  0.51802   Mean     :  0.53293
## 3rd Qu.:2013-01-08   3rd Qu.:  1.10792   3rd Qu.:   1.15150
## Max.    :2025-09-01   Max.    :  9.26768   Max.     : 10.79841
```

```
cor(dat$g1, dat$g2)
```

```
## [1] 0.9617833
```

```
dat$Month <- yearmonth(dat$Date)
dat_ts <- as_tsibble(dat, index = Month)
```

```
fit <- model(
  dat_ts,
  var = VAR(vars(g1, g2) ~ AR(p = 1:6), ic = "bic")
)

report(fit)
```

```
## Series: g1, g2
## Model: VAR(6)
##
## Coefficients for g1:
##      lag(g1,1) lag(g2,1) lag(g1,2) lag(g2,2) lag(g1,3) lag(g2,3)
##      -0.3555   1.7797   3.9526   -5.4028   -7.9026   9.3101
## s.e.    0.6750   0.6627   1.8120   1.7802   2.4177   2.3830
##      lag(g1,4) lag(g2,4) lag(g1,5) lag(g2,5) lag(g1,6) lag(g2,6)
##      10.8388  -11.8567  -9.8039   10.5326   4.2191  -4.3851
## s.e.    2.4097   2.3788   1.7788   1.7575   0.6521   0.6447
##
## Coefficients for g2:
##      lag(g1,1) lag(g2,1) lag(g1,2) lag(g2,2) lag(g1,3) lag(g2,3)
##      -2.9141   4.3111   6.5283   -7.9594   -9.3905   10.7901
## s.e.    0.6876   0.6750   1.8457   1.8134   2.4628   2.4274
##      lag(g1,4) lag(g2,4) lag(g1,5) lag(g2,5) lag(g1,6) lag(g2,6)
##      11.6765  -12.6828  -10.3300   11.0552   4.4139  -4.5736
## s.e.    2.4546   2.4232   1.8119   1.7903   0.6643   0.6567
##
## Residual covariance matrix:
##      g1      g2
## g1 0.2295 0.2333
## g2 0.2333 0.2381
##
## log likelihood = 872.97
## AIC = -1689.95   AICc = -1687.11 BIC = -1566.74
```

The two samples used are the MSA rates in Oxnard and LA. They both have 609 observations, which were taken monthly since 1975.

Looking at the above data (descriptive and forecasting), we can see that the growth rate in MSA1 (Oxnard) is 0.52%, very similar to that of MSA2 (LA), which is 0.53%. They two sets are very correlated (0.96).

Using the fpp3 package's var() function, chosen using BIC to keep models simple, we find that a var(6) model leads to the most accurate results. BIC is minimized at p=6 which is why var(6) was chosen. In this model, almost all of the lag coefficients are significant, implying a strong correlation between prior lags and current lags. There are also clear ties between Oxnard rates and LA rates.

11.2

```
p <- 6
n <- nrow(dat)
idx <- (p + 1):n

Y1 <- dat$g1[idx] # Oxnard_t
```

```

Y2 <- dat$g2[idx] # LA_t

X <- NULL
for (i in 1:p) {
  X <- cbind(X, dat$g1[idx - i], dat$g2[idx - i])
}
X <- as.data.frame(X)
names(X) <- unlist(lapply(1:p, function(i) c(paste0("g1_L", i), paste0("g2_L", i))))

df <- data.frame(Y1 = Y1, Y2 = Y2, X)

fit1_u <- lm(Y1 ~ ., data = df)
fit2_u <- lm(Y2 ~ ., data = df)

cols_g1 <- grep("^g1_L", names(df), value = TRUE)
cols_g2 <- grep("^g2_L", names(df), value = TRUE)

fit1_r <- lm(Y1 ~ ., data = df[, c("Y1", cols_g1), drop = FALSE])
fit2_r <- lm(Y2 ~ ., data = df[, c("Y2", cols_g2), drop = FALSE])

gc_g2_to_g1 <- anova(fit1_r, fit1_u)
gc_g1_to_g2 <- anova(fit2_r, fit2_u)

gc_g2_to_g1

```

```

## Analysis of Variance Table
##
## Model 1: Y1 ~ g1_L1 + g1_L2 + g1_L3 + g1_L4 + g1_L5 + g1_L6
## Model 2: Y1 ~ Y2 + g1_L1 + g2_L1 + g1_L2 + g2_L2 + g1_L3 + g2_L3 + g1_L4 +
##          g2_L4 + g1_L5 + g2_L5 + g1_L6 + g2_L6
##   Res.Df    RSS Df Sum of Sq    F    Pr(>F)
## 1      595 152.162
## 2      588   0.486   7    151.68 26203 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```
gc_g1_to_g2
```

```

## Analysis of Variance Table
##
## Model 1: Y2 ~ g2_L1 + g2_L2 + g2_L3 + g2_L4 + g2_L5 + g2_L6
## Model 2: Y2 ~ Y1 + g1_L1 + g2_L1 + g1_L2 + g2_L2 + g1_L3 + g2_L3 + g1_L4 +
##          g2_L4 + g1_L5 + g2_L5 + g1_L6 + g2_L6
##   Res.Df    RSS Df Sum of Sq    F    Pr(>F)
## 1      595 168.319
## 2      588   0.504   7    167.81 27947 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

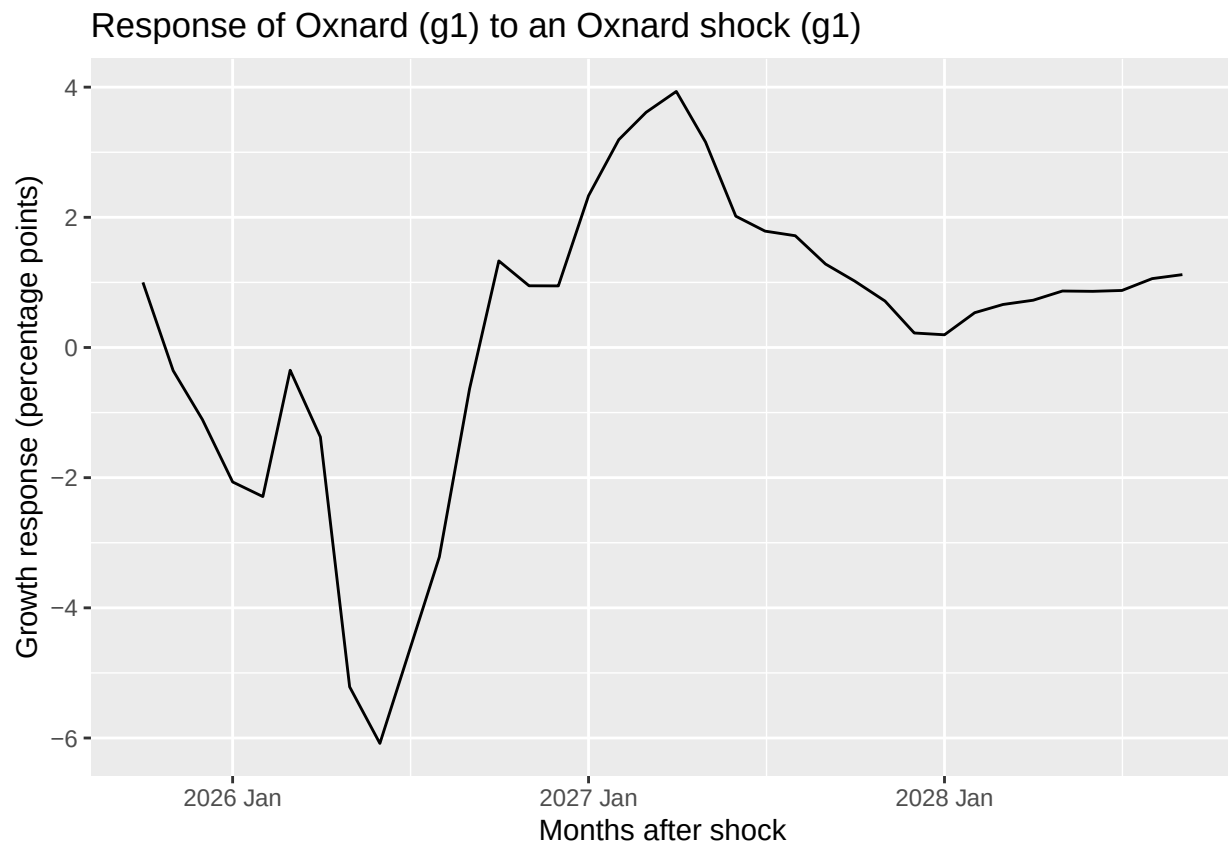
Looking at the Granger Causality results, we see that regressing y2 on y1 and y1 on y2 both yield significant p values (sub 5%). This means that LA Granger-causes Oxnard MSA and Oxnard Granger-causes LA MSA. In both cases, we can reject the “no cause” null hypothesis.

11.3

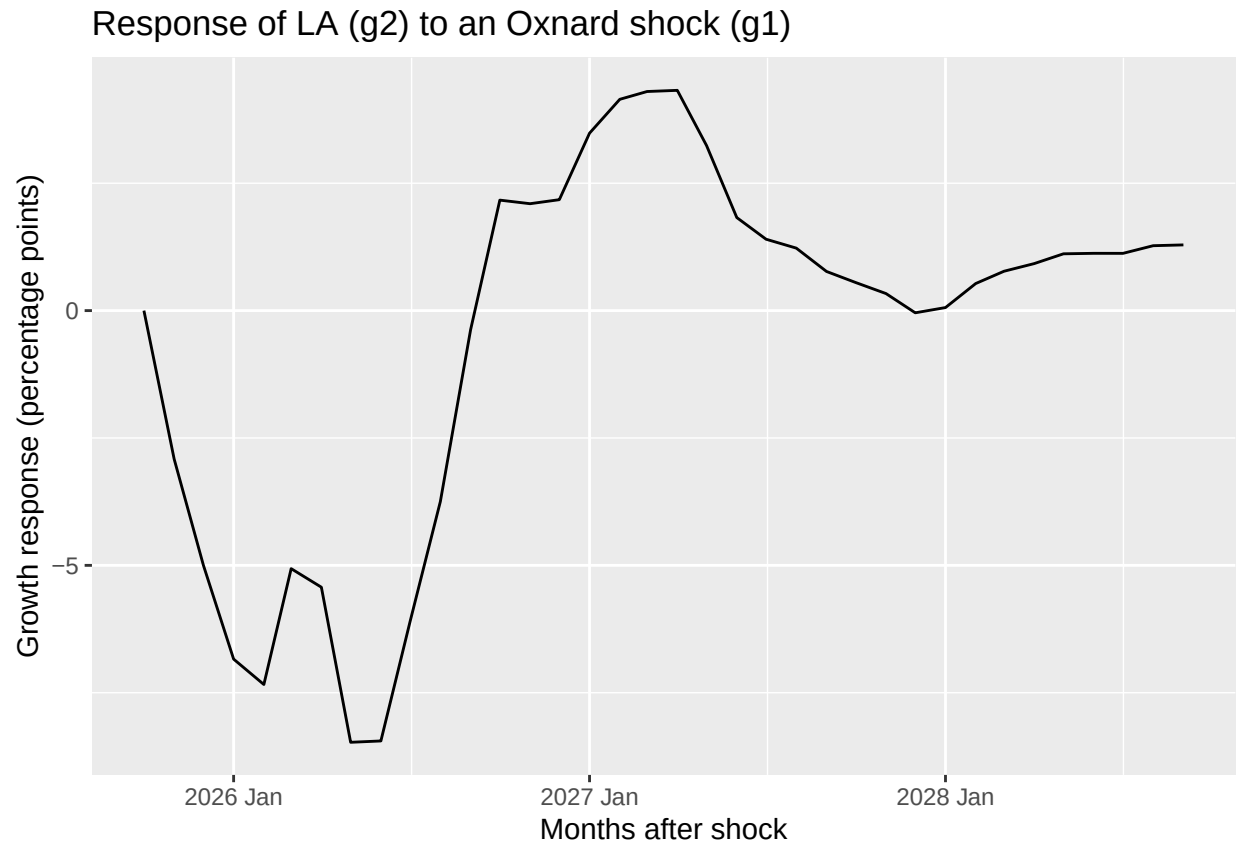
```
h <- 36
future <- new_data(dat_ts, n = h)

irf_g1 <- IRF(fit, new_data = future, impulse = "g1")
irf_g2 <- IRF(fit, new_data = future, impulse = "g2")

autoplot(irf_g1, .vars = g1) +
  labs(
    title = "Response of Oxnard (g1) to an Oxnard shock (g1)",
    x = "Months after shock",
    y = "Growth response (percentage points)"
  )
```

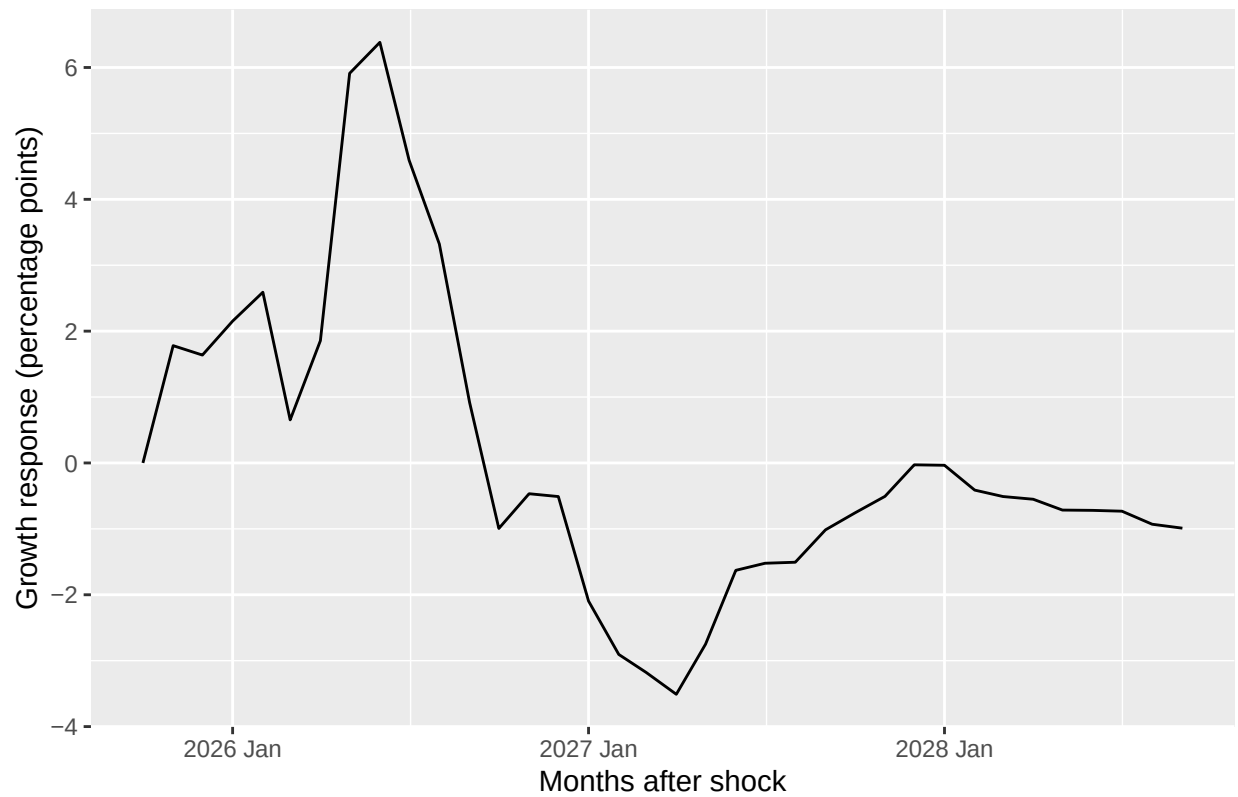


```
autoplot(irf_g1, .vars = g2) +
  labs(
    title = "Response of LA (g2) to an Oxnard shock (g1)",
    x = "Months after shock",
    y = "Growth response (percentage points)"
  )
```

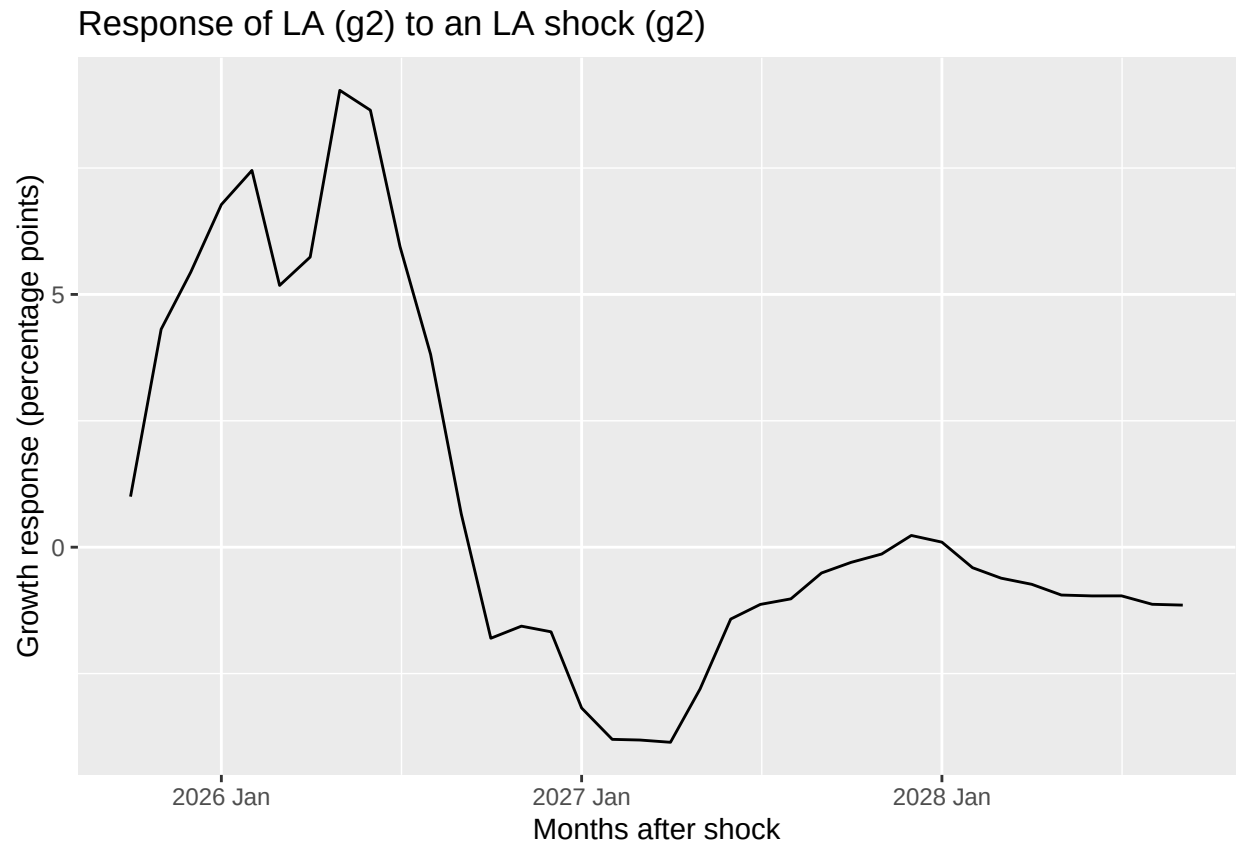



```
autoplot(irf_g2, .vars = g1) +  
  labs(  
    title = "Response of Oxnard (g1) to an LA shock (g2)",  
    x = "Months after shock",  
    y = "Growth response (percentage points)"  
  )
```

Response of Oxnard (g1) to an LA shock (g2)



```
autoplot(irf_g2, .vars = g2) +  
  labs(  
    title = "Response of LA (g2) to an LA shock (g2)",  
    x = "Months after shock",  
    y = "Growth response (percentage points)"  
  )
```



The above graphs show that in forecasted data, shocks are always positive. LA shocks are more positive than Oxnard shocks, which tend to lead to an immediate downturn before spiking 6 months later, but shocks in both cities lead to overall positive effects roughly 6-12 months out. The resulting growth rates return to 0 after 2 years.

Given my prior knowledge of the regions (growing up in LA and then moving to over the summer Oxnard), these results seem in line. I was surprised to see such similar growth rates since 1975, but the effects of shocks make sense to me. When LA grows, competing markets also grow, and vice versa.

The graph shows that order does not affect my results. Although the graphs change a bit, general trends and spikes stay exactly the same.

Textbook C: Hyndman's Book (2nd Ed)

Sourced from <https://otexts.com/fpp2/>

7.7

```
library(fma)
```

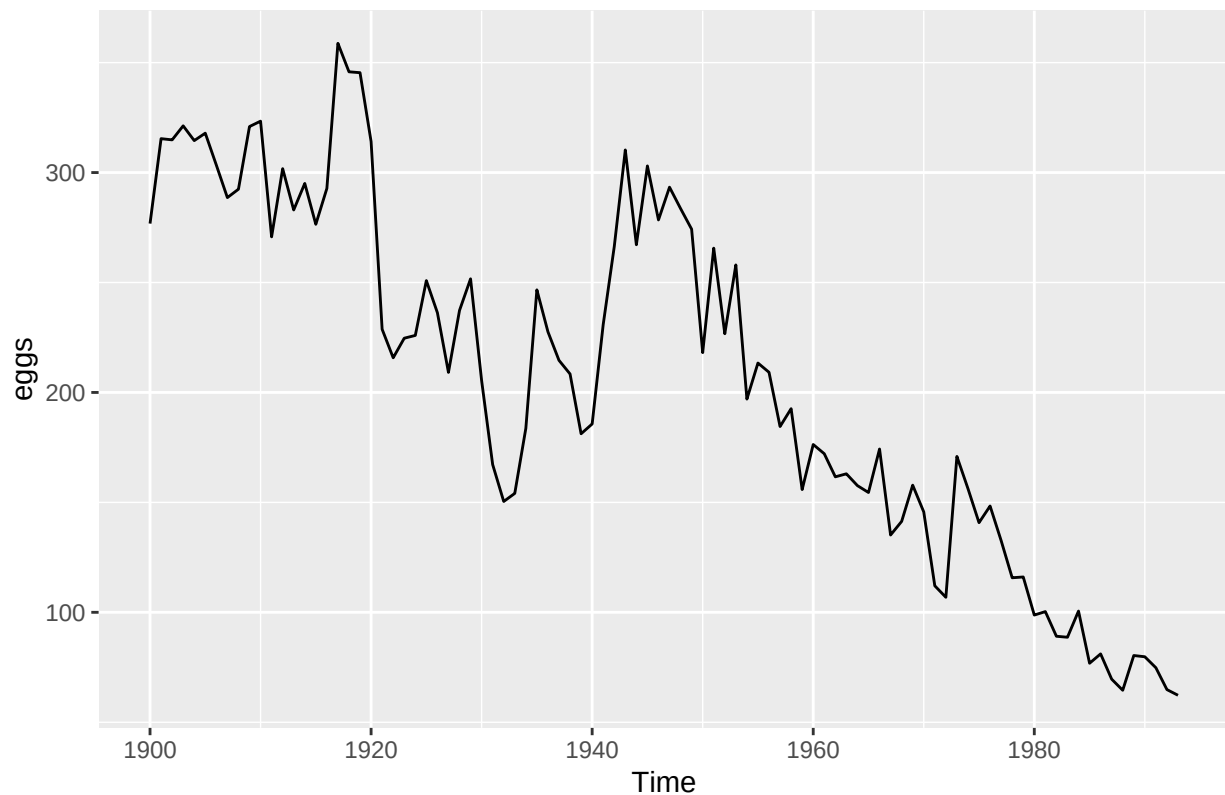
```
##
## Attaching package: 'fma'
```

```
## The following objects are masked from 'package:astsa':  
##  
##   chicken, sales  
  
## The following objects are masked from 'package:DAAG':  
##  
##   milk, ozone  
  
## The following objects are masked from 'package:MASS':  
##  
##   cement, housing, petrol
```

```
head(eggs)
```

```
## Time Series:  
## Start = 1900  
## End = 1905  
## Frequency = 1  
## [1] 276.79 315.42 314.87 321.25 314.54 317.92
```

```
autoplot(eggs)
```



```

h <- 100

fit1 <- holt(eggs, h = h)

fit2 <- holt(eggs, damped = TRUE, h = h)

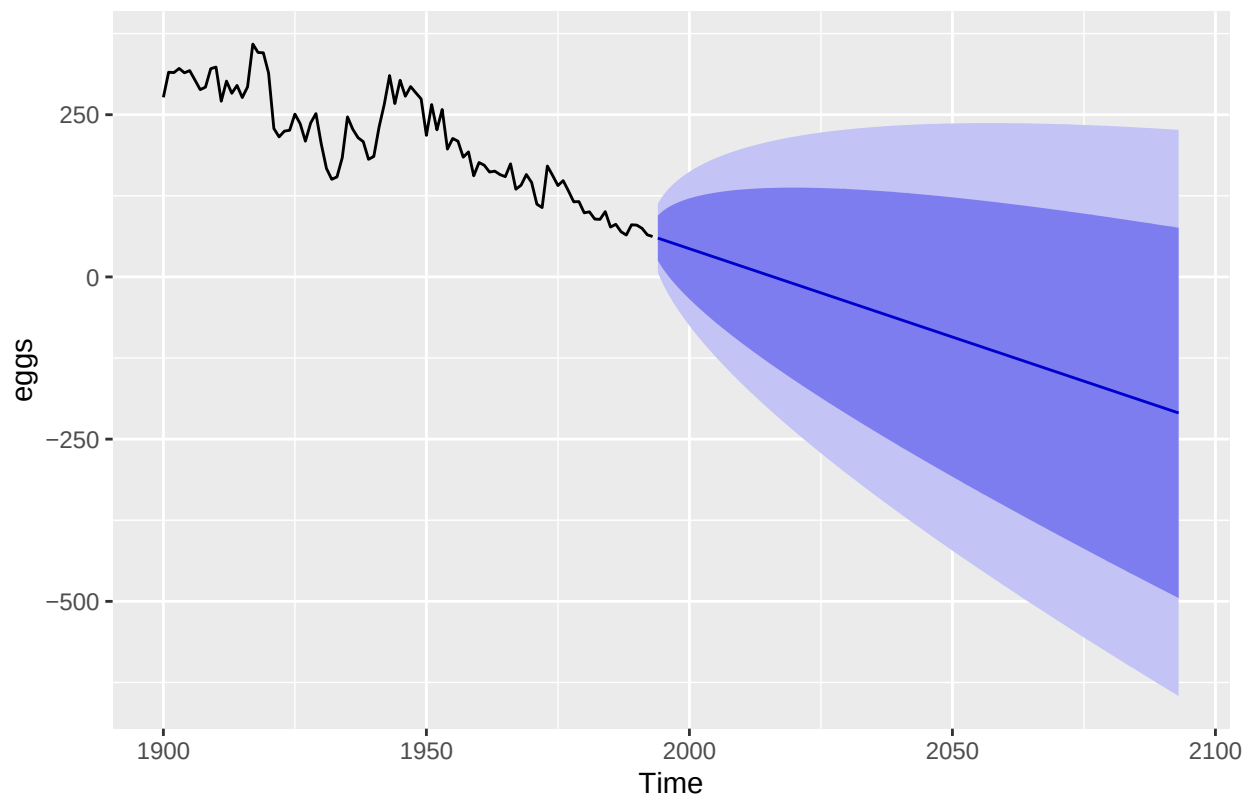
lambda <- BoxCox.lambda(eggs)
fit3 <- holt(eggs, lambda = lambda, h = h)

fit4 <- holt(eggs, damped = TRUE, lambda = lambda, h = h)

autoplot(fit1)

```

Forecasts from Holt's method

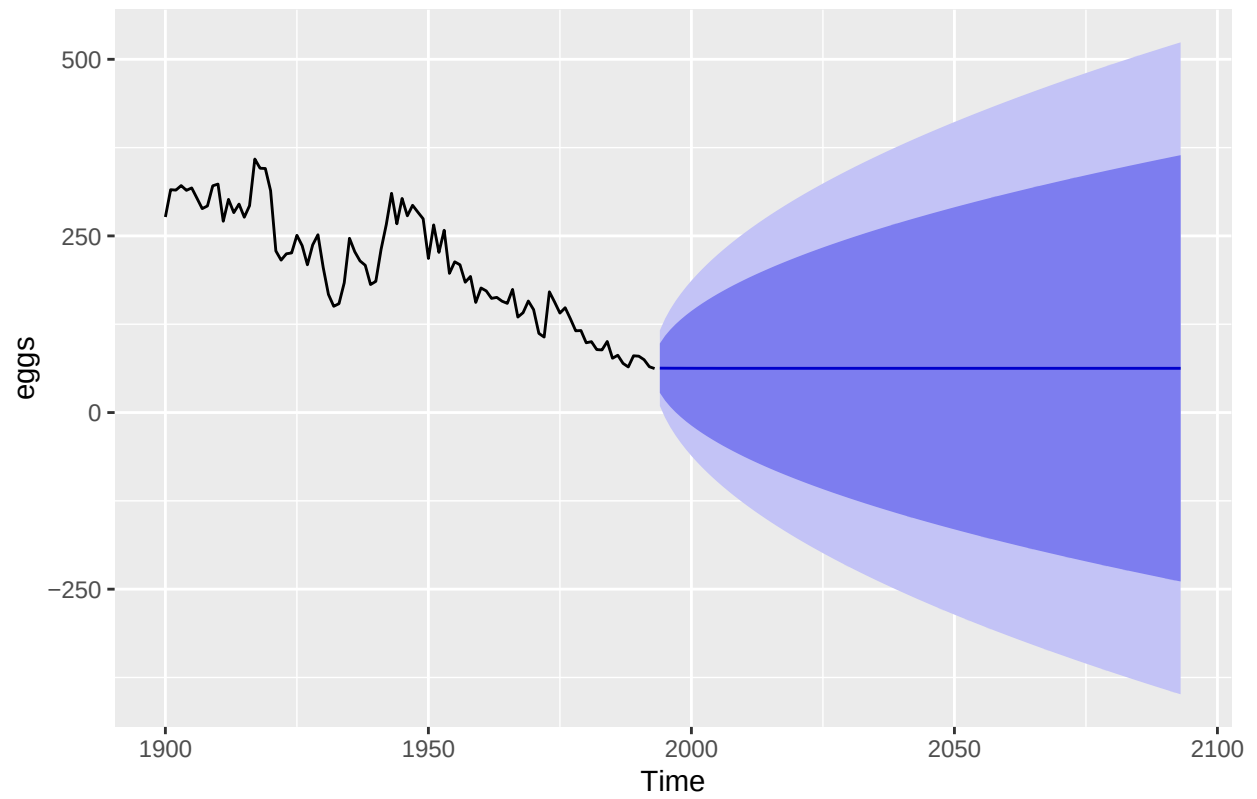


```

autoplot(fit2)

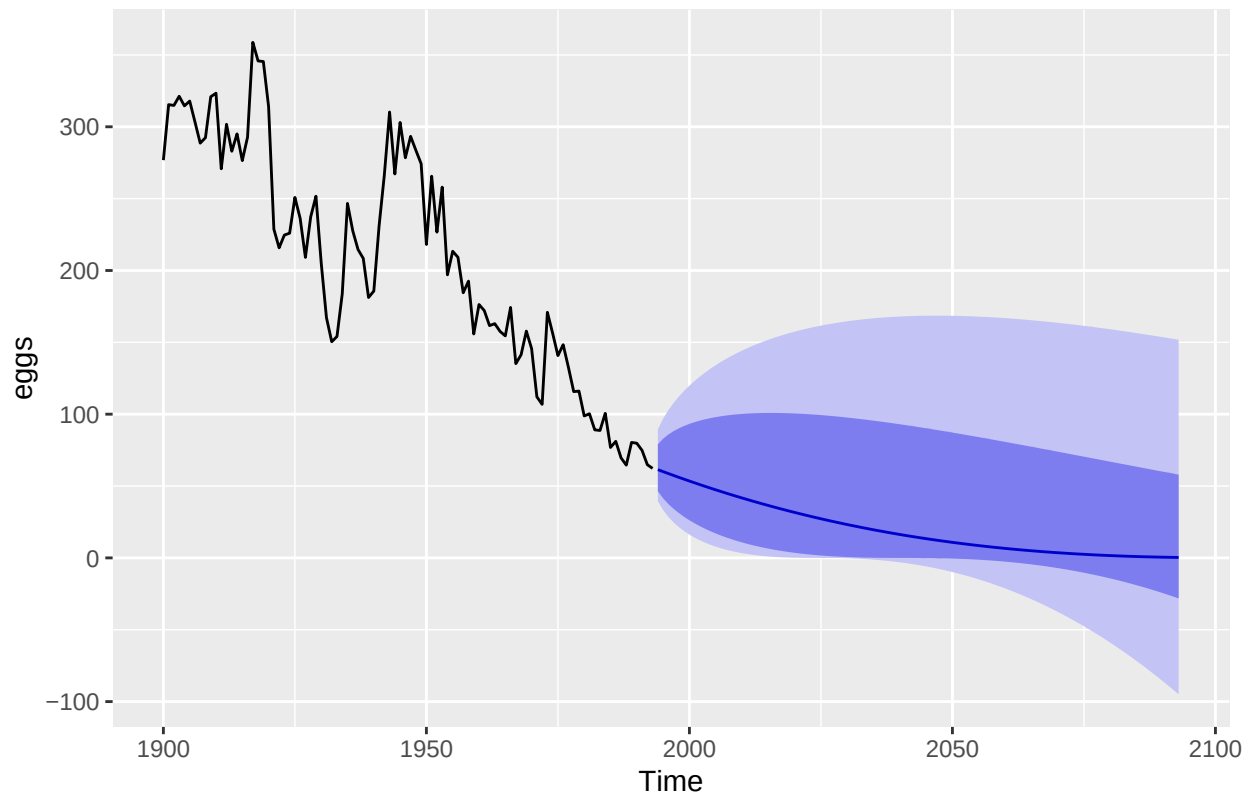
```

Forecasts from Damped Holt's method



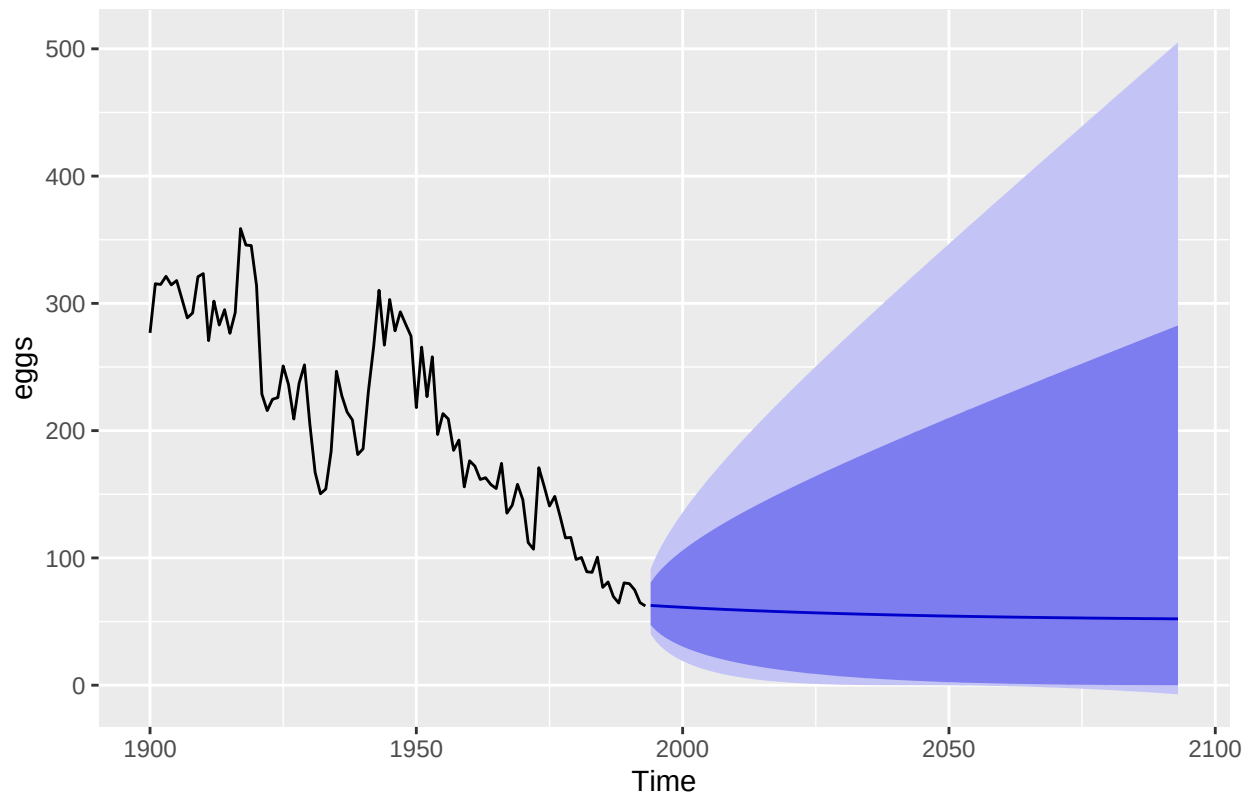
```
autoplot(fit3)
```

Forecasts from Holt's method



```
autoplot(fit4)
```

Forecasts from Damped Holt's method



```
rmse1 <- sqrt(mean(residuals(fit1)^2, na.rm = TRUE))
rmse2 <- sqrt(mean(residuals(fit2)^2, na.rm = TRUE))
rmse3 <- sqrt(mean(residuals(fit3)^2, na.rm = TRUE))
rmse4 <- sqrt(mean(residuals(fit4)^2, na.rm = TRUE))
```

```
c(no_trans      = rmse1,
   damped        = rmse2,
   boxcox        = rmse3,
   boxcox_damped = rmse4)
```

```
##      no_trans      damped      boxcox boxcox_damped
##      26.582188      26.540185      1.032217      1.039187
```

Looking at the 4 models, it looks like the box-cox method and the box-cox with a dampening affect have much lower RMSEs than the first two models. The box-cox (no dampening) is slightly lower, so fit3 is the best model by RMSE, but the difference is negligible so you can't say for certain that fit3 is the best predictive model.

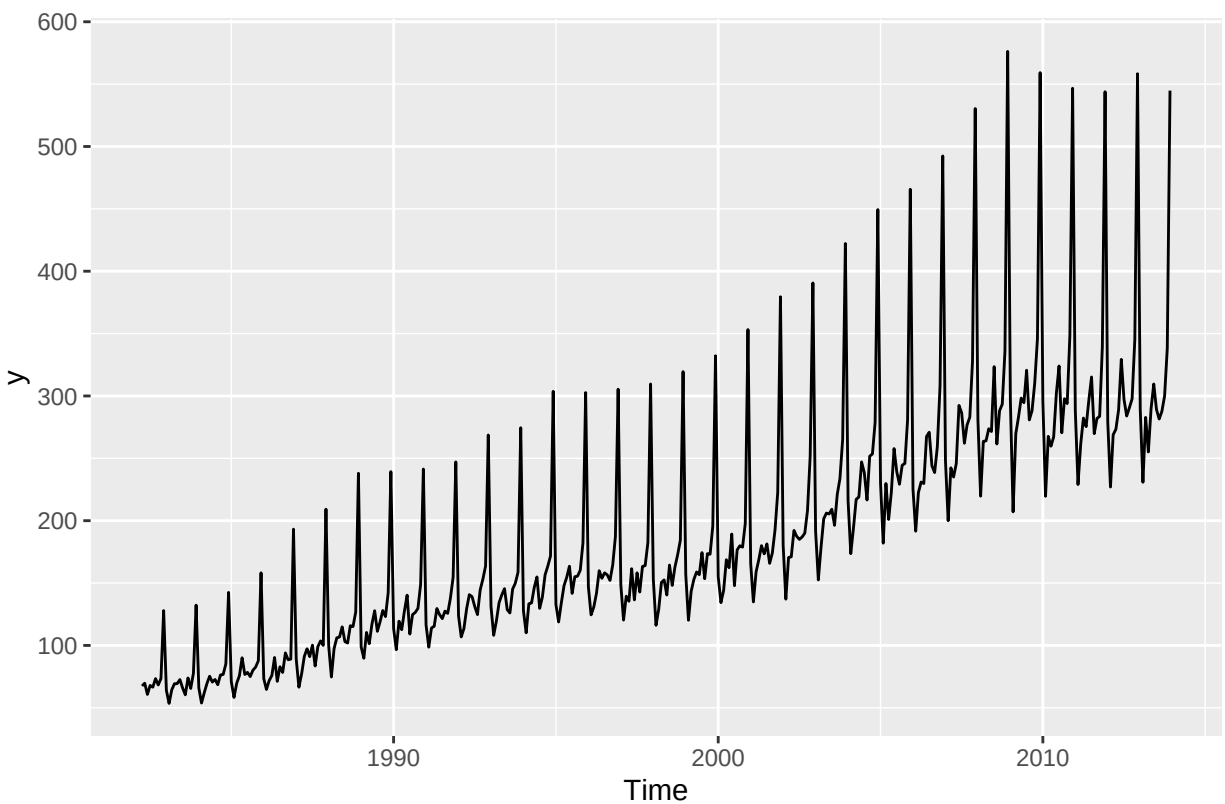
7.8

2.3 Data


```
retaildata <- readxl::read_excel("retail.xlsx", skip=1)

y <- ts(retaildata[, "A3349562T"],
        frequency=12, start=c(1982,4))

autoplot(y)
```



a)

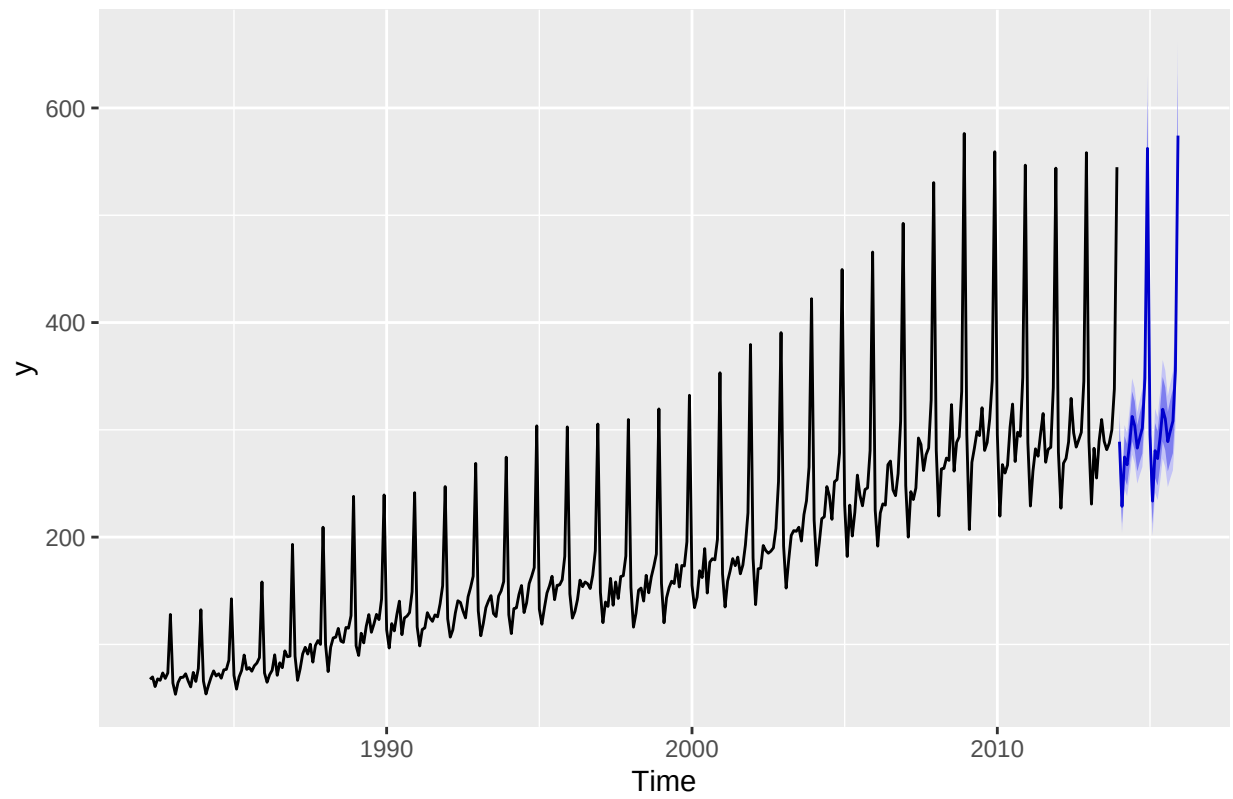
Multiplicative seasonality is necessary due to the size of the swings. They tend to be bigger when the series level is higher and as time goes by. Since seasonal trends aren't constant, we cannot use additive seasonality here and must use multiplicative.

b)

```
fit_hw <- hw(y, seasonal = "multiplicative") # normal trend
fit_hwd <- hw(y, seasonal = "multiplicative", damped = TRUE) # damped trend

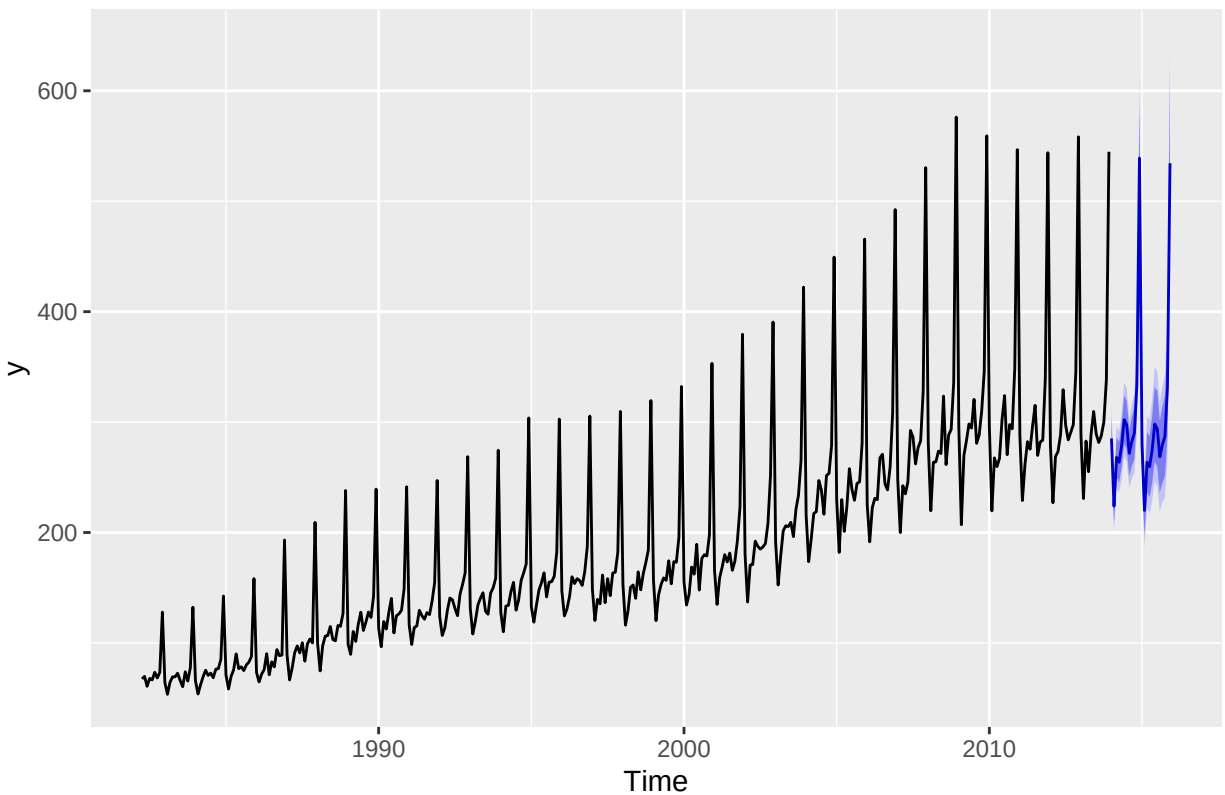
autoplot(fit_hw)
```

Forecasts from Holt–Winters' multiplicative method



```
autoplot(fit_hwd)
```

Forecasts from Damped Holt–Winters' multiplicative method



Using a Damped Holt-Winters forecast slightly decreases the forecasted results.

c)

```
rmse_hw  <- sqrt(mean(residuals(fit_hw)^2, na.rm = TRUE))  
rmse_hwd <- sqrt(mean(residuals(fit_hwd)^2, na.rm = TRUE))
```

```
rmse_hw
```

```
## [1] 0.05236847
```

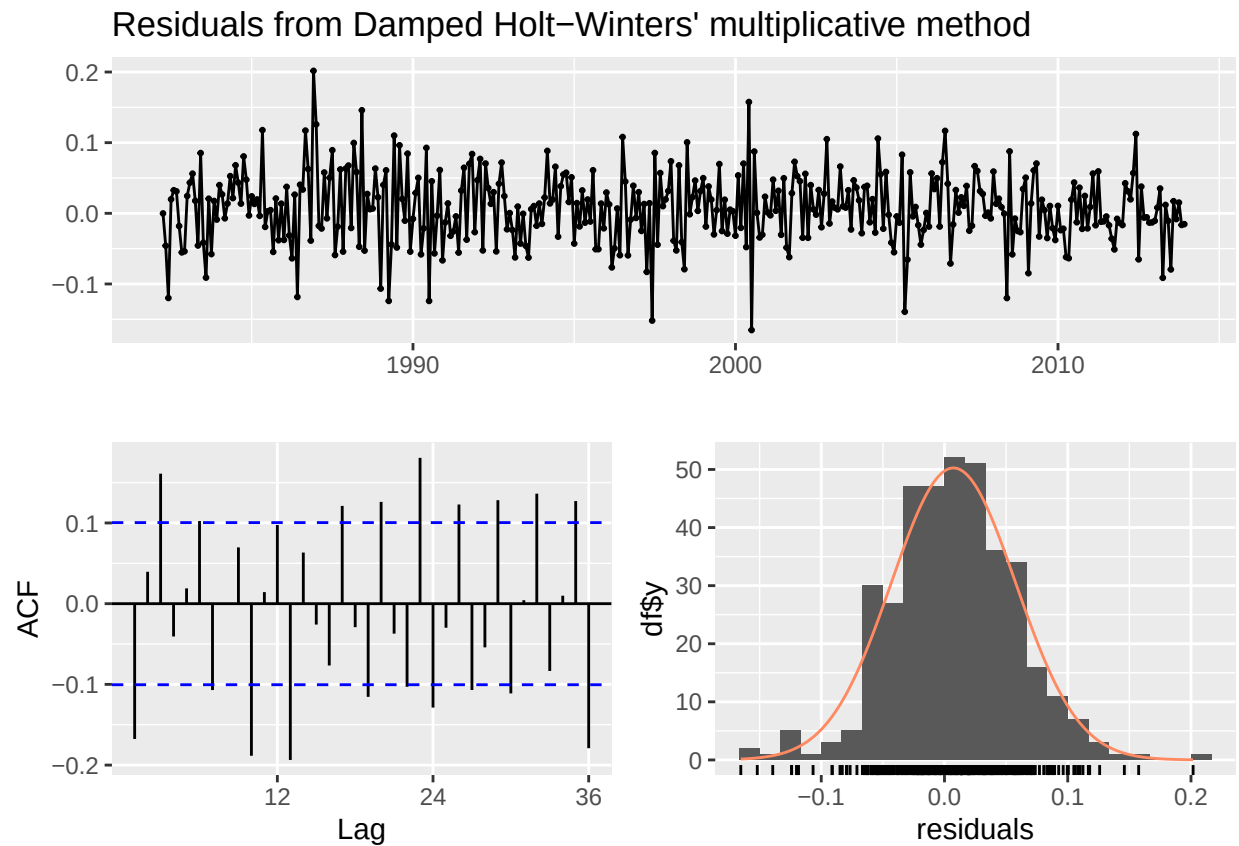
```
rmse_hwd
```

```
## [1] 0.05090279
```

The Damped method has a slightly lower RMSE, so by that metric we prefer the damped method of forecasting.

d)

```
checkresiduals(fit_hwd)
```



```
##  
##  Ljung-Box test  
##  
## data:  Residuals from Damped Holt-Winters' multiplicative method  
## Q* = 112.5, df = 24, p-value = 2.061e-13  
##  
## Model df: 0.   Total lags used: 24
```

The residuals visually appear to be white noise. The time series of residuals is completely random, the residuals are normally distributed, and the ACF plot does not show a clear pattern.

The tells Ljung-Box test tells a different story. The p value is well below 0.05, which tells us there still might be some autocorrelation in the model.

e)

```
y_train <- window(y, end = c(2010, 12))  
y_test  <- window(y, start = c(2011, 1))  
  
h_test <- length(y_test)
```

```
fc_best <- hw(y_train, seasonal = "multiplicative", damped = TRUE, h = h_test)
fc_snaive <- snaive(y_train, h = h_test)

rmse_best_test <- sqrt(mean((y_test - fc_best$mean)^2))
rmse_snaive_test <- sqrt(mean((y_test - fc_snaive$mean)^2))

rmse_best_test
```

```
## [1] 12.64729
```

```
rmse_snaive_test
```

```
## [1] 13.66134
```

Looking at the RMSE from our best model against the naive forecasting method used in 3.7, we prefer our best model. The RMSE is almost 10% lower.

7.9

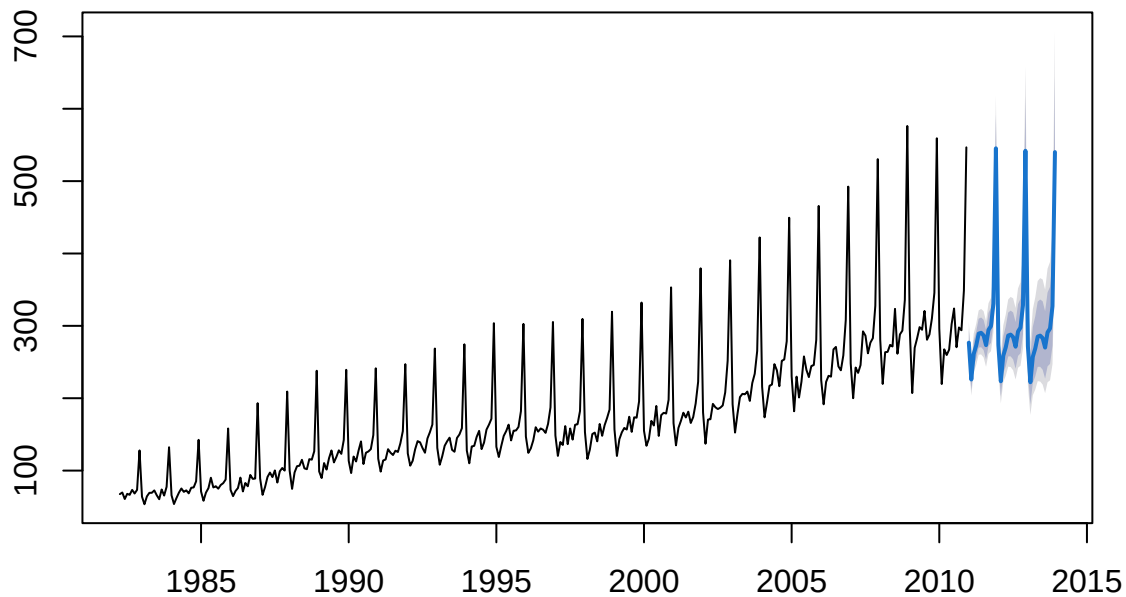
```
y_train <- window(y, end = c(2010, 12))
y_test <- window(y, start = c(2011, 1))
h_test <- length(y_test)

lambda <- BoxCox.lambda(y_train)

fit_stl_ets <- stlm(y_train,
  lambda = lambda,
  s.window = "periodic",
  method = "ets")

fc_stl_ets <- forecast(fit_stl_ets, h = h_test)
plot(fc_stl_ets)
```

Forecasts from STL + ETS(A,Ad,N)



```
rmse_stl_ets_test <- sqrt(mean((y_test - fc_stl_ets$mean)^2))
rmse_stl_ets_test
```

```
## [1] 13.58208
```

The RMSE value from the STL decomp applied to the Box-Cox transformed series is lower than the naive method but still not as low as the best method from 7.8.

7.11

Data

```
library(fpp2)
```

```
## -- Attaching packages ----- fpp2 2.5 --
```

```
## v expsmooth 2.3
```

```
## -- Conflicts ----- fpp2_conflicts --
```

```
## x forecast::easter() masks tis::easter()
```

```
## x forecast::getResponse() masks nlme::getResponse()
```

```
##
## Attaching package: 'fpp2'

## The following object is masked from 'package:fpp3':
##
##   insurance

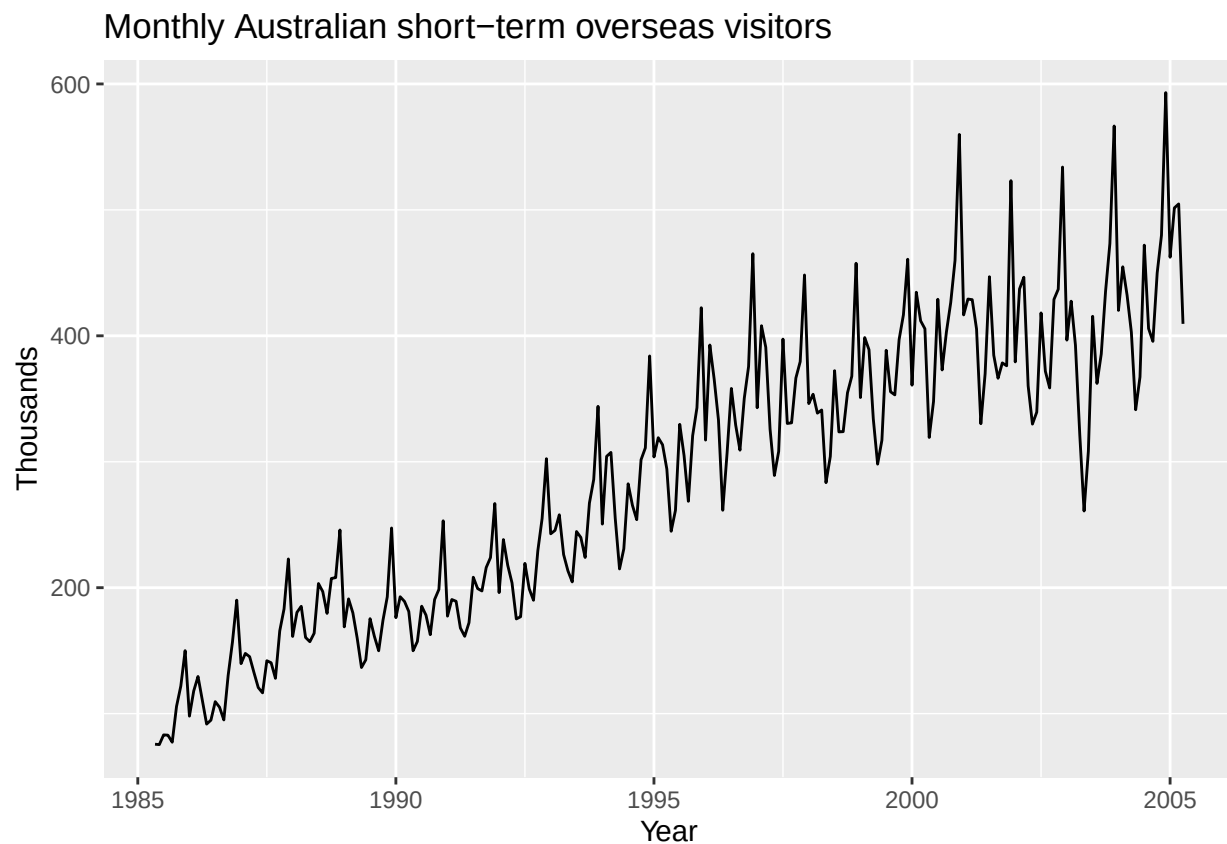
## The following object is masked from 'package:astsa':
##
##   oil
```

```
head(visitors)
```

```
##      May   Jun   Jul   Aug   Sep   Oct
## 1985  75.7  75.4  83.1  82.9  77.3 105.7
```

a)

```
autoplot(visitors) +
  ggtitle("Monthly Australian short-term overseas visitors") +
  xlab("Year") + ylab("Thousands")
```



This series has an upwards trend and a consistent seasonal pattern. At a glance, there doesn't appear to be a strong cyclical pattern.

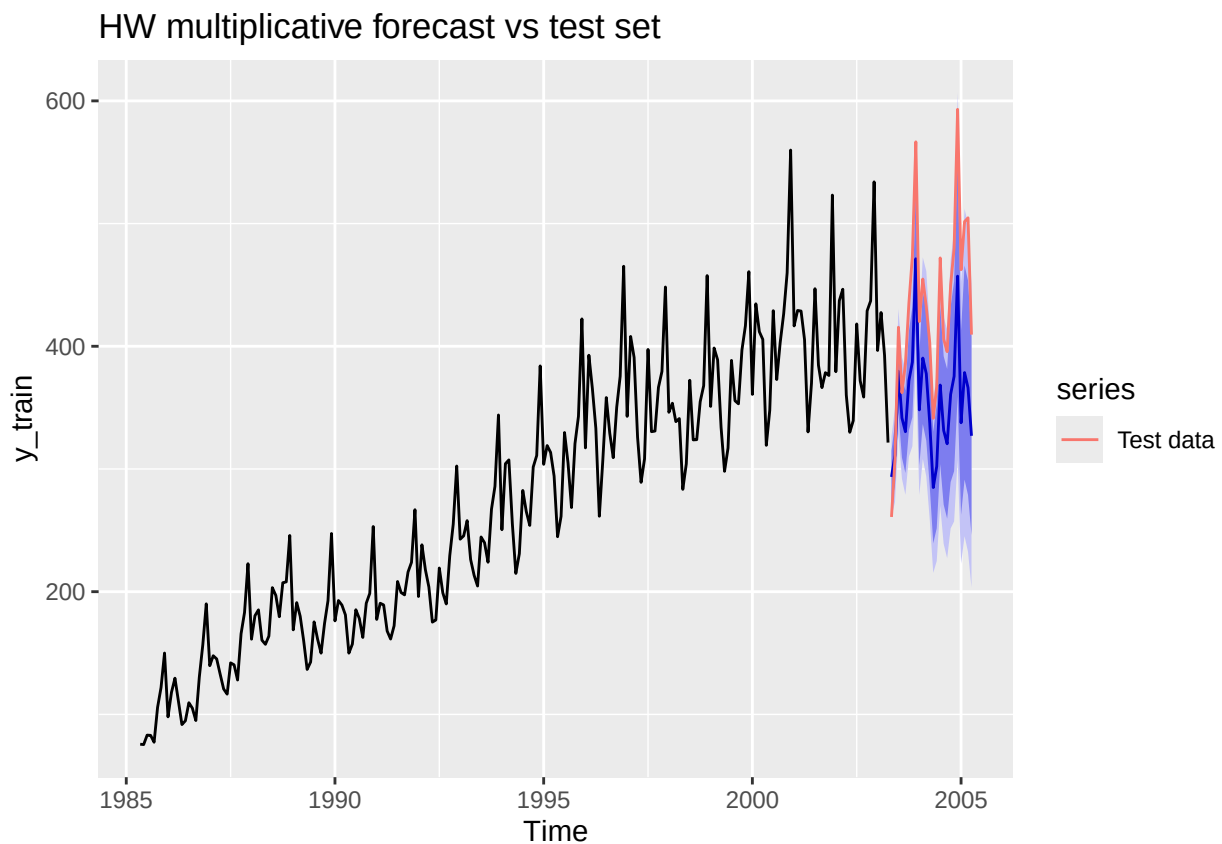
b)

```
y <- visitors

y_train <- window(y, end = c(2003, 4))
y_test  <- window(y, start = c(2003, 5))
h_test  <- length(y_test)

fit_hw_mul <- hw(y_train, seasonal = "multiplicative", h = h_test)

autoplot(fit_hw_mul) +
  autolayer(y_test, series = "Test data") +
  ggtitle("HW multiplicative forecast vs test set")
```



```
rmse_hw_mul <- sqrt(mean((y_test - fit_hw_mul$mean)^2))
rmse_hw_mul
```

```
## [1] 83.23541
```

The above plot shows our forecasted data (blue) is consistently underpredicting the actual data (red). The RMSE is 83.2, which doesn't mean much on its own but will help compare this forecast to other forecasts.

c)

The seasonal effects seem to get larger (more stretched out on graph) as the trend increases, implying a multiplicative pattern over a seasonal pattern. It is not as clear as with 7.8.a, but it is multiplicative nevertheless.

d)

```
y <- visitors
y_train <- window(y, end = c(2003, 4))
y_test <- window(y, start = c(2003, 5))
h_test <- length(y_test)

lambda <- BoxCox.lambda(y_train)

## (i)
fit_ets <- ets(y_train)
fc_ets <- forecast(fit_ets, h = h_test)

## (ii)
fit_ets_box <- ets(y_train, model = "AAA", lambda = lambda, biasadj = TRUE)
fc_ets_box <- forecast(fit_ets_box, h = h_test)

## (iii)
fc_snaive <- snaive(y_train, h = h_test)

## (iv)
fit_stl_ets <- stlm(y_train,
                    lambda = lambda,
                    s.window = "periodic",
                    method = "ets")
fc_stl_ets <- forecast(fit_stl_ets, h = h_test)

rmse_ets <- sqrt(mean((y_test - fc_ets$mean)^2))
rmse_ets_box <- sqrt(mean((y_test - fc_ets_box$mean)^2))
rmse_snaive <- sqrt(mean((y_test - fc_snaive$mean)^2))
rmse_stl_ets <- sqrt(mean((y_test - fc_stl_ets$mean)^2))

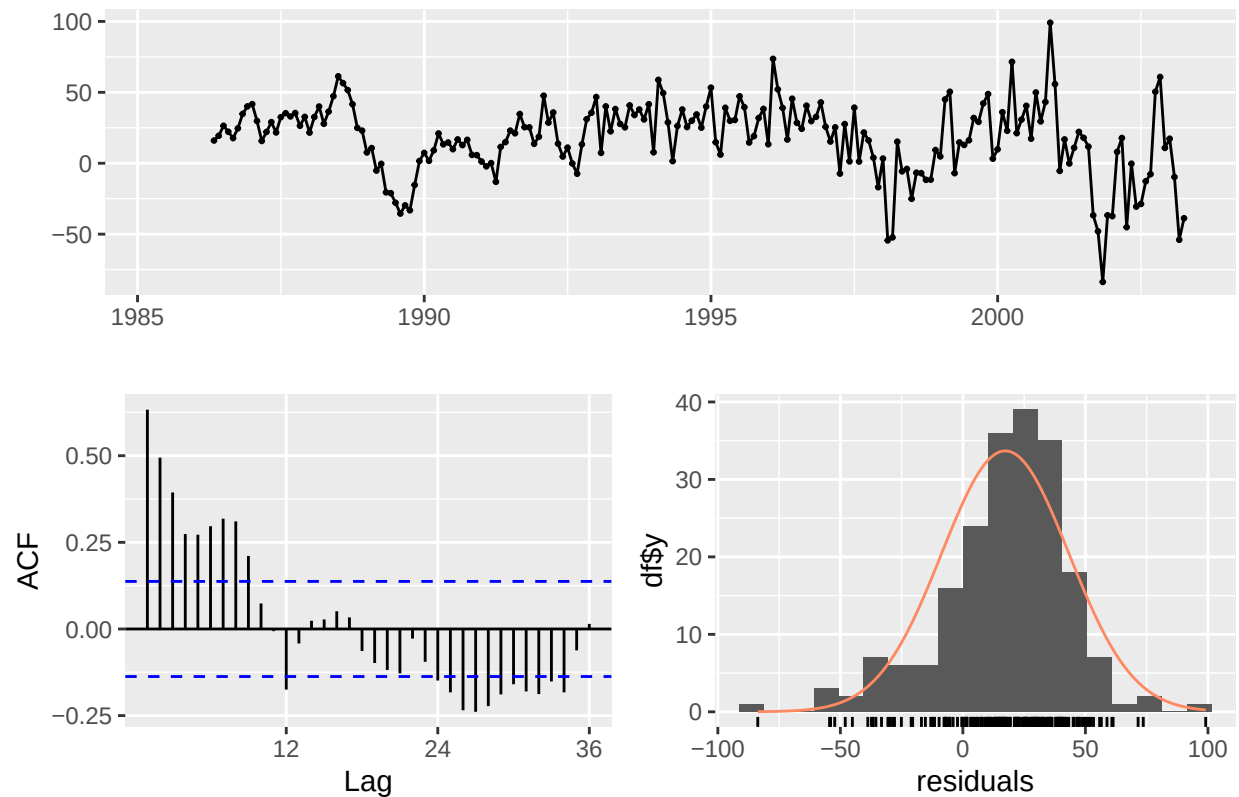
c(ETS = rmse_ets,
  ETS_BoxCox_AAA = rmse_ets_box,
  Seasonal_naive = rmse_snaive,
  STL_ETS_BoxCox = rmse_stl_ets)
```

```
##          ETS ETS_BoxCox_AAA Seasonal_naive STL_ETS_BoxCox
##      80.23124      77.27352      50.30097      53.23069
```

e)

```
checkresiduals(fc_snaive)
```

Residuals from Seasonal naive method



```
##
##  Ljung-Box test
##
## data:  Residuals from Seasonal naive method
## Q* = 295.02, df = 24, p-value < 2.2e-16
##
## Model df: 0.   Total lags used: 24
```

The seasonal naive method has the lowest RMSE, making it the best model. The residuals are not white noise, and seem to suggest some added cyclical trends that weren't picked up. The residuals plot isn't random, the ACF has spikes (maybe an AR(4) or AR(8) would help), and the residuals barplot is skewed.

f)

```
library(forecast)

y <- visitors

lambda <- BoxCox.lambda(y)

f_ets <- function(y, h) {
  forecast(ets(y), h = h)
}
```

```

f_ets_box <- function(y, h) {
  forecast(ets(y, model = "AAA", lambda = lambda, biasadj = TRUE), h = h)
}

f_snaive <- function(y, h) {
  snaive(y, h = h)
}

f_stl_ets <- function(y, h) {
  forecast(
    stlm(y, lambda = lambda, s.window = "periodic", method = "ets"),
    h = h
  )
}

cv_ets      <- tsCV(y, f_ets,      h = 1)
cv_ets_box  <- tsCV(y, f_ets_box, h = 1)
cv_snaive   <- tsCV(y, f_snaive,  h = 1)
cv_stl_ets  <- tsCV(y, f_stl_ets, h = 1)

rmse_cv_ets      <- sqrt(mean(cv_ets^2,      na.rm = TRUE))
rmse_cv_ets_box  <- sqrt(mean(cv_ets_box^2,  na.rm = TRUE))
rmse_cv_snaive   <- sqrt(mean(cv_snaive^2,   na.rm = TRUE))
rmse_cv_stl_ets  <- sqrt(mean(cv_stl_ets^2,  na.rm = TRUE))

c(
  CV_ETS      = rmse_cv_ets,
  CV_ETS_BoxCox_AAA = rmse_cv_ets_box,
  CV_Seasonal_naive = rmse_cv_snaive,
  CV_STL_ETS_BoxCox = rmse_cv_stl_ets
)

```

```

##          CV_ETS CV_ETS_BoxCox_AAA CV_Seasonal_naive CV_STL_ETS_BoxCox
##      18.52985      19.02610      32.62379      18.18674

```

In this test, the ETS, ETS Box-Cox, and the STL ETS Box-Cox all have very similarly low RMSE values. STL ETS Box-Cox is lowest, followed by ETS and then ETS Box-Cox. The Seasonal naive method is now the worst one by far.

8.13

Data

```

library(fpp2)
head(qcement)

```

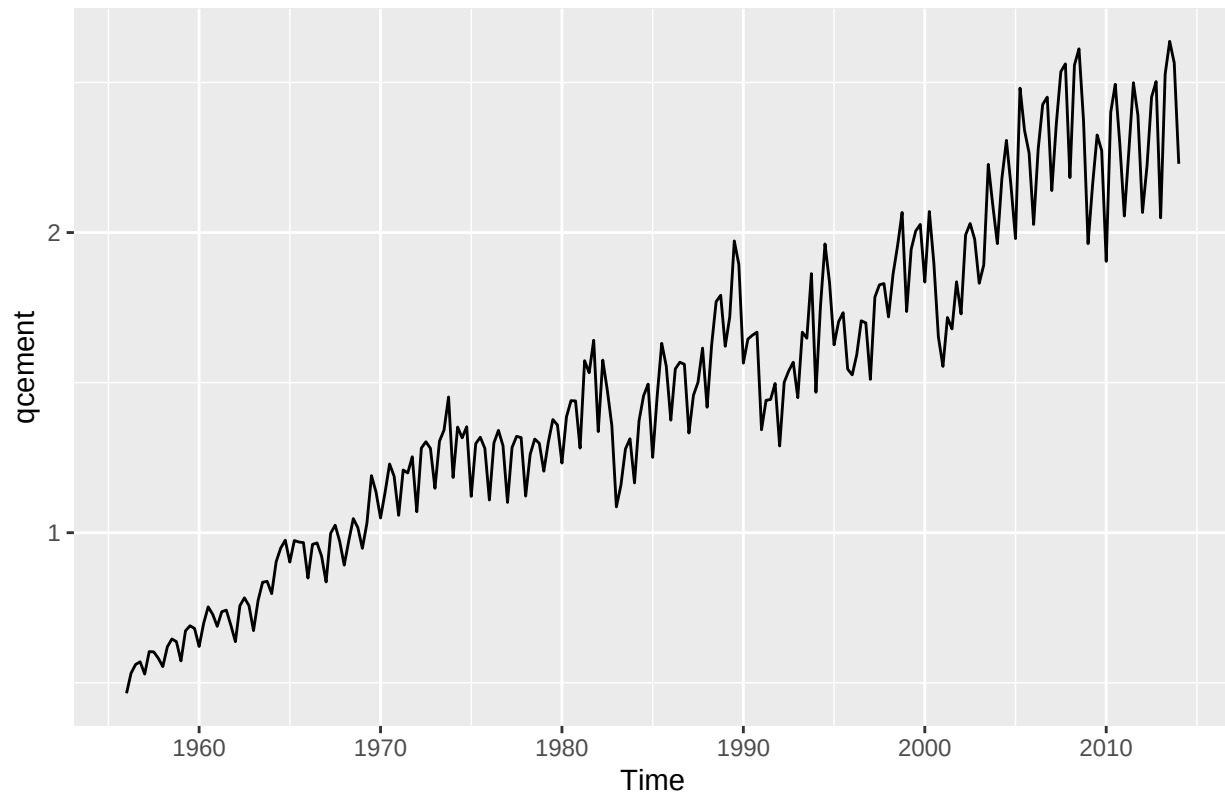
```

##      Qtr1 Qtr2 Qtr3 Qtr4
## 1956 0.465 0.532 0.561 0.570
## 1957 0.529 0.604

```

a)

```
autoplot(qcement)
```



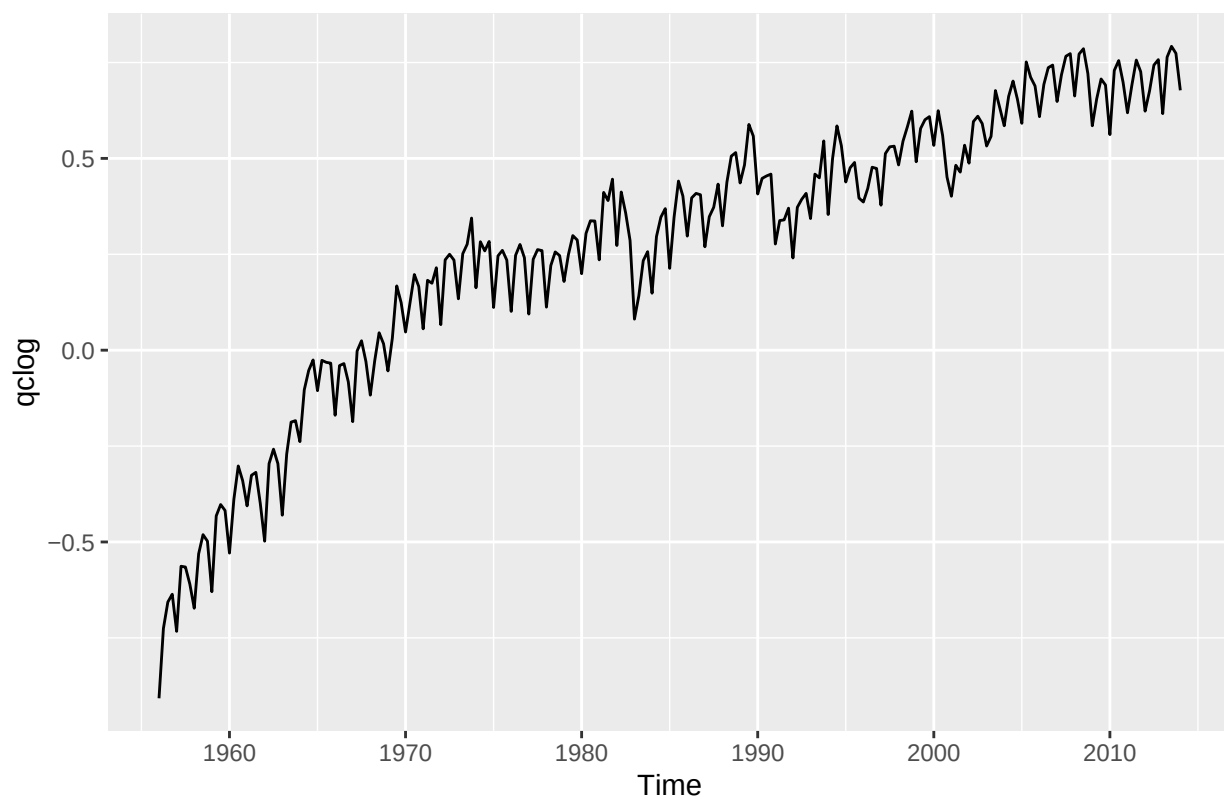
```
lambda_qc <- BoxCox.lambda(qcement)
lambda_qc
```

```
## [1] -0.4319725
```

Because the variance changes with each level, a Box-Cox transformation is appropriate. This is further supported by lambda being fairly close to 0 (-0.43).

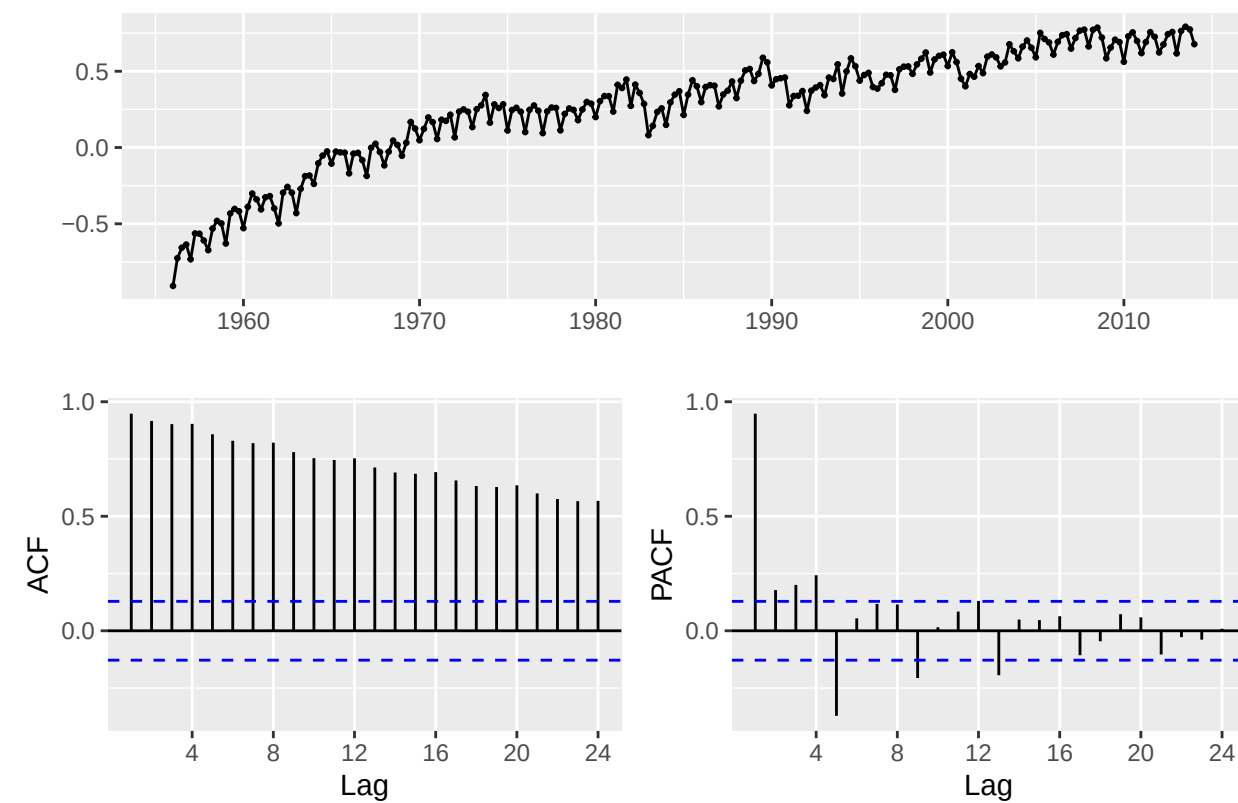
b)

```
qclog <- BoxCox(qcement, lambda_qc)
autoplot(qclog)
```

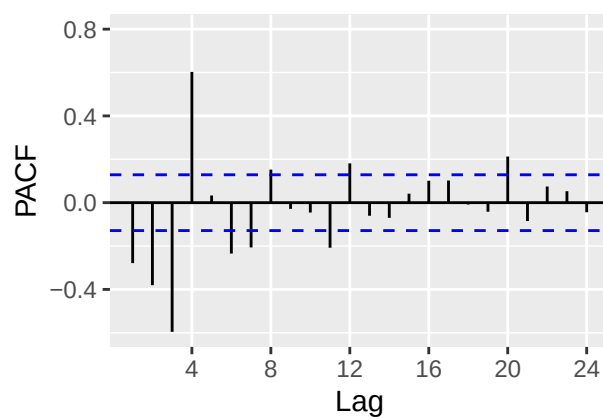
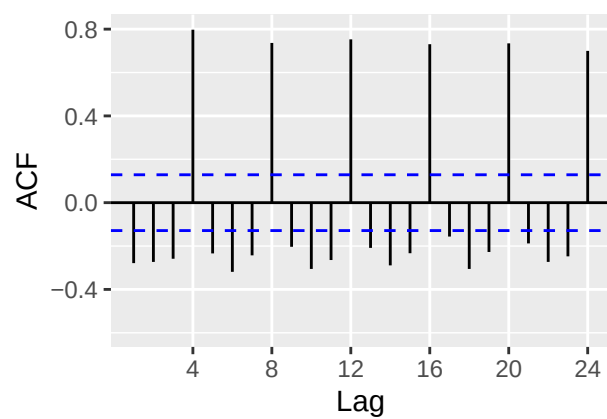
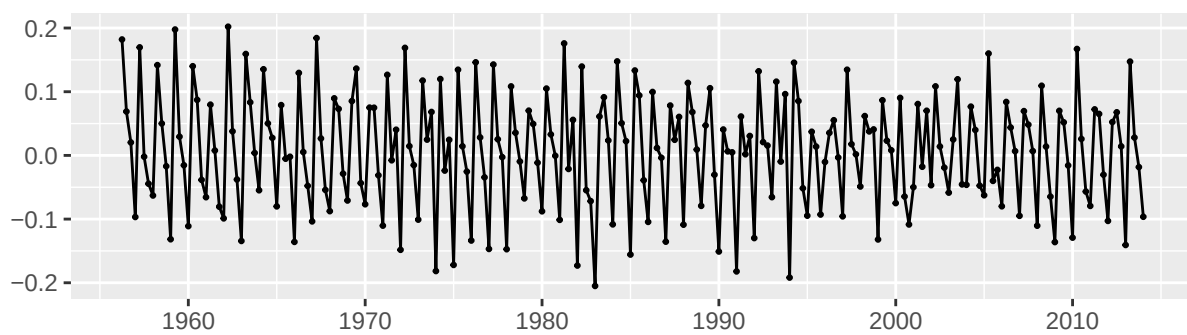


```
ggtsdisplay(qclog)
```

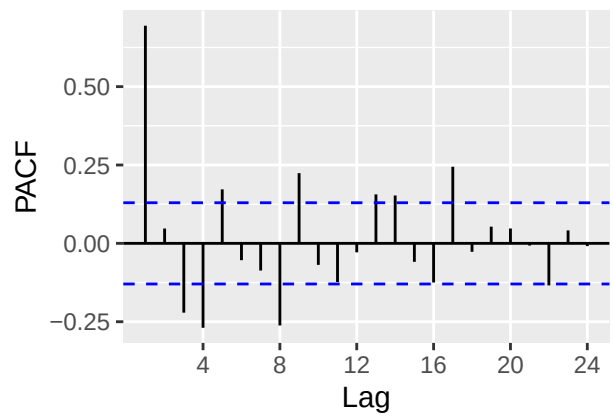
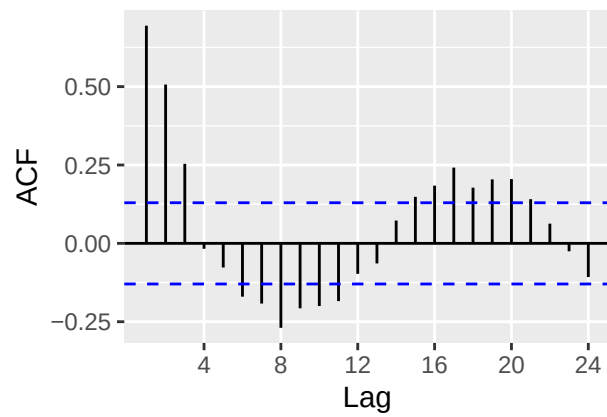
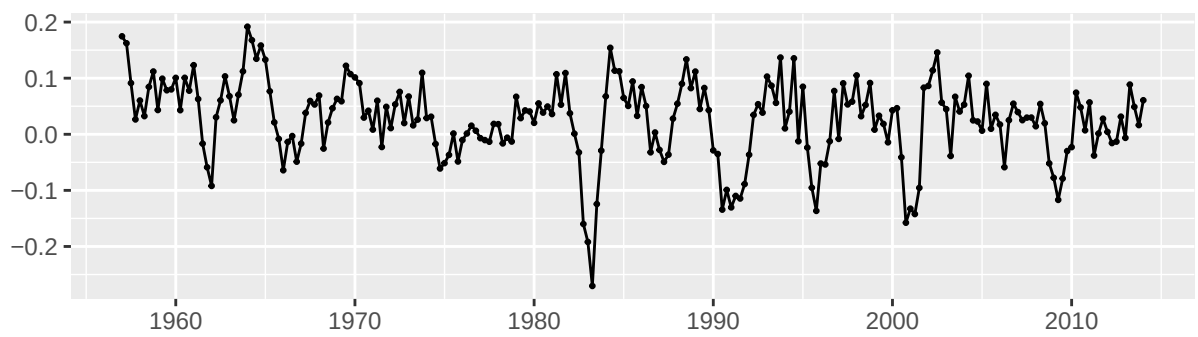
```
# raw
```



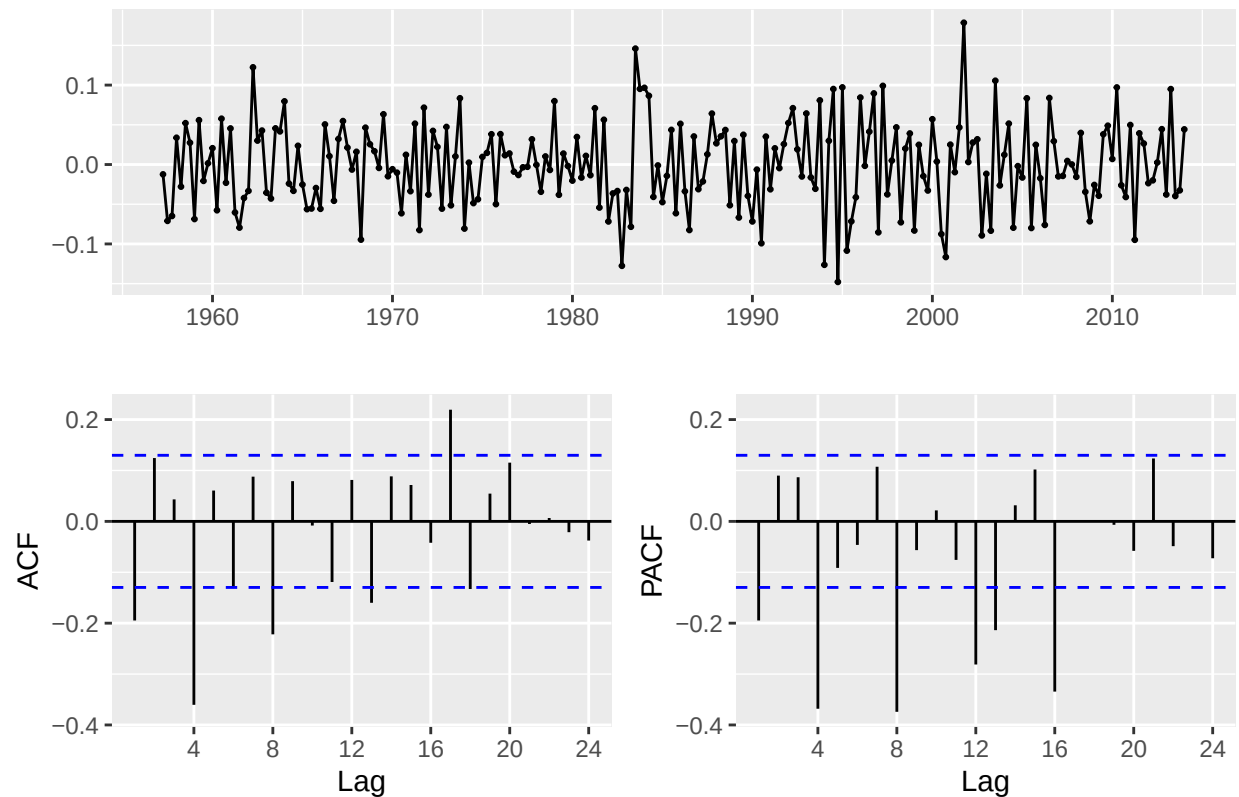
```
ggtsdisplay(diff(qclog))          # non-seasonal diff
```



```
ggtsdisplay(diff(qclog, lag = 4))      # seasonal diff
```



```
ggtsdisplay(diff(diff(qclog), lag = 4)) # both
```

The data is not stationary at first, but once you apply seasonal and non-seasonal differences, the dataset is properly cleaned and stationary.

c)

```
library(forecast)

fit1 <- Arima(qclog, order = c(0,1,1),
              seasonal = c(0,1,1))

fit2 <- Arima(qclog, order = c(1,1,1),
              seasonal = c(0,1,1))

fit3 <- Arima(qclog, order = c(0,1,2),
              seasonal = c(0,1,1))

AIC(fit1)
```

```
## [1] -799.675
```

```
AIC(fit2)
```

```
## [1] -802.7888
```

```
AIC(fit3)
```

```
## [1] -801.6818
```

When it comes to AIC, you always want to choose the lowest value. In our case, that leads us to pick fit2, which is an ARIMA(1,1,1)(0,1,1)[4] model.

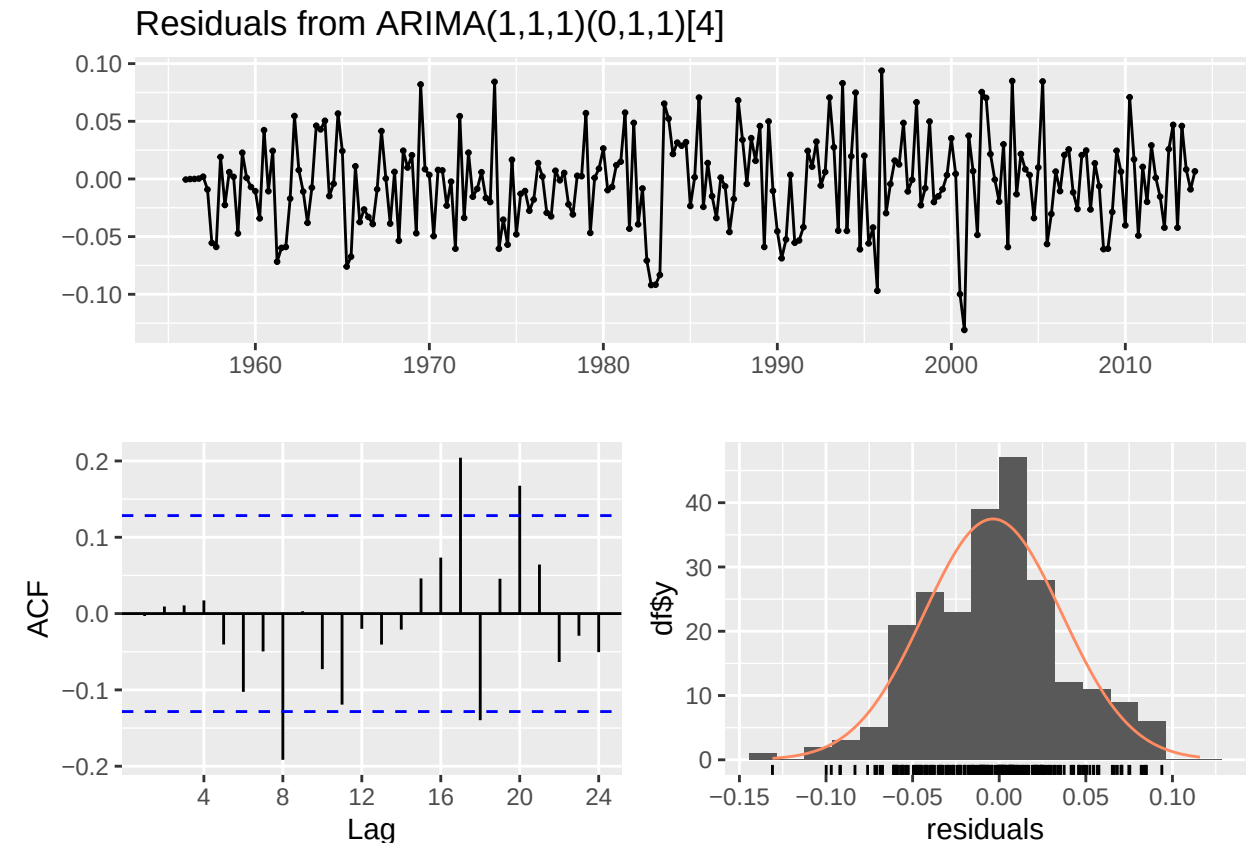
d)

```
best_fit <- fit2
```

```
summary(best_fit)
```

```
## Series: qclog
## ARIMA(1,1,1)(0,1,1)[4]
##
## Coefficients:
##          ar1      ma1      sma1
##      -0.5946  0.3772 -0.8855
## s.e.   0.1689  0.1915  0.0358
##
## sigma^2 = 0.00165: log likelihood = 405.39
## AIC=-802.79  AICc=-802.61  BIC=-789.07
##
## Training set error measures:
##              ME      RMSE      MAE      MPE      MAPE      MASE
## Training set -0.003631088 0.03991437 0.03109056 -7.592297 24.87064 0.5169697
##              ACF1
## Training set -0.00318632
```

```
checkresiduals(best_fit)
```



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1)(0,1,1)[4]
## Q* = 12.583, df = 5, p-value = 0.02762
##
## Model df: 3.    Total lags used: 8

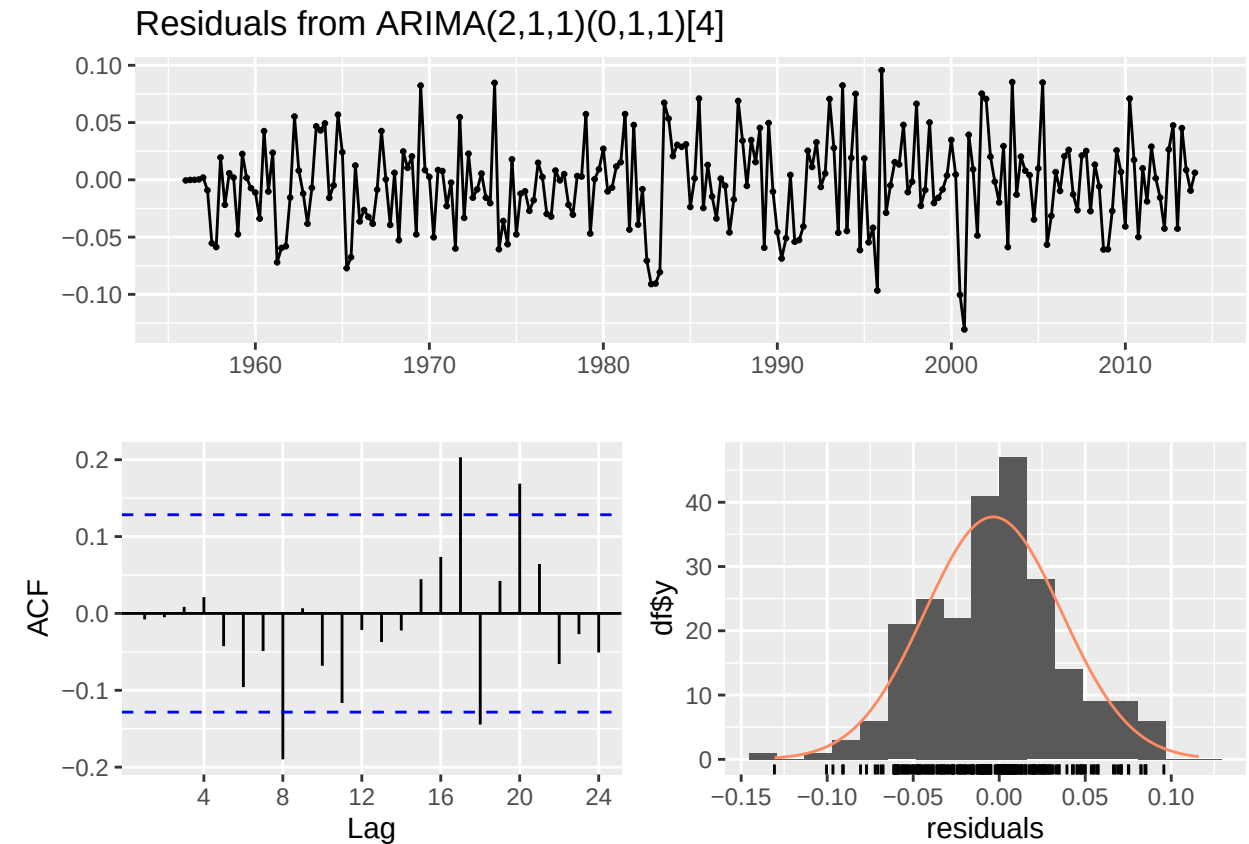
fit_new <- Arima(qclog, order = c(2,1,1),
                 seasonal = c(0,1,1))    # ARIMA(2,1,1)(0,1,1)[4]

summary(fit_new)

## Series: qclog
## ARIMA(2,1,1)(0,1,1)[4]
##
## Coefficients:
##          ar1      ar2      ma1      sma1
##       -0.4578  0.0457  0.2457 -0.8825
## s.e.   0.5588  0.1626  0.5560   0.0379
##
## sigma^2 = 0.001657:  log likelihood = 405.43
## AIC=-800.87   AICc=-800.6   BIC=-783.72
##
## Training set error measures:
```

```
##               ME          RMSE          MAE          MPE          MAPE          MASE
## Training set -0.003511989 0.03991199 0.03112158 -7.779875 25.13652 0.5174855
##               ACF1
## Training set -0.0078145
```

```
checkresiduals(fit_new) # p-value should be larger; residuals closer to white noise
```



```
##
## Ljung-Box test
##
## data: Residuals from ARIMA(2,1,1)(0,1,1)[4]
## Q* = 12.145, df = 4, p-value = 0.0163
##
## Model df: 4. Total lags used: 8
```

```
## Compare AICs
AIC(best_fit); AIC(fit_new)
```

```
## [1] -802.7888
```

```
## [1] -800.8681
```

The estimated coefficients for this model are -0.5946 (AR(1)), 0.3772 (MA(1)), and -0.8855 (SMA(1)). Looking at the residuals, the time series is almost completely white noise shaped, the residuals are normal, and the ACF has a few tiny spikes but seems to indicate that all seasonal patterns have been accounted for.

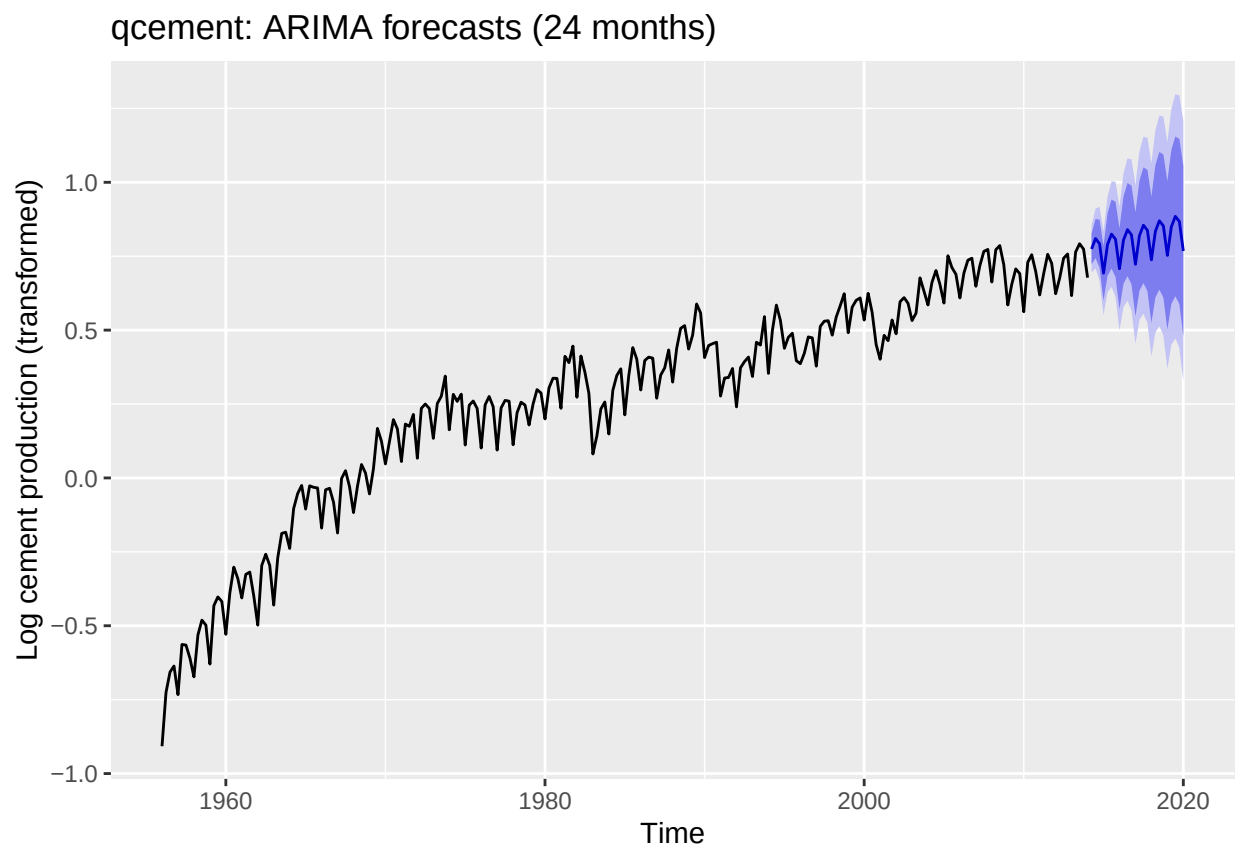
The Ljung-Box test has a p value of 0.02, which means we reject the null hypothesis. There is still some autocorrelation in our model.

Trying out a new Arima model doesn't make our model better by AIC.

e)

```
fc_arima <- forecast(best_fit, h = 24)

autoplot(fc_arima) +
  ggtitle("qcement: ARIMA forecasts (24 months)") +
  ylab("Log cement production (transformed)")
```



The next 24 observations continue the pattern, showing additive seasonality and a slight upwards trend. The prediction interval widens, which means we are less certain of the prediction the further away we are.

f)

```
fit_ets <- ets(qclog)
summary(fit_ets)
```

```
## ETS(A,Ad,A)
##
```

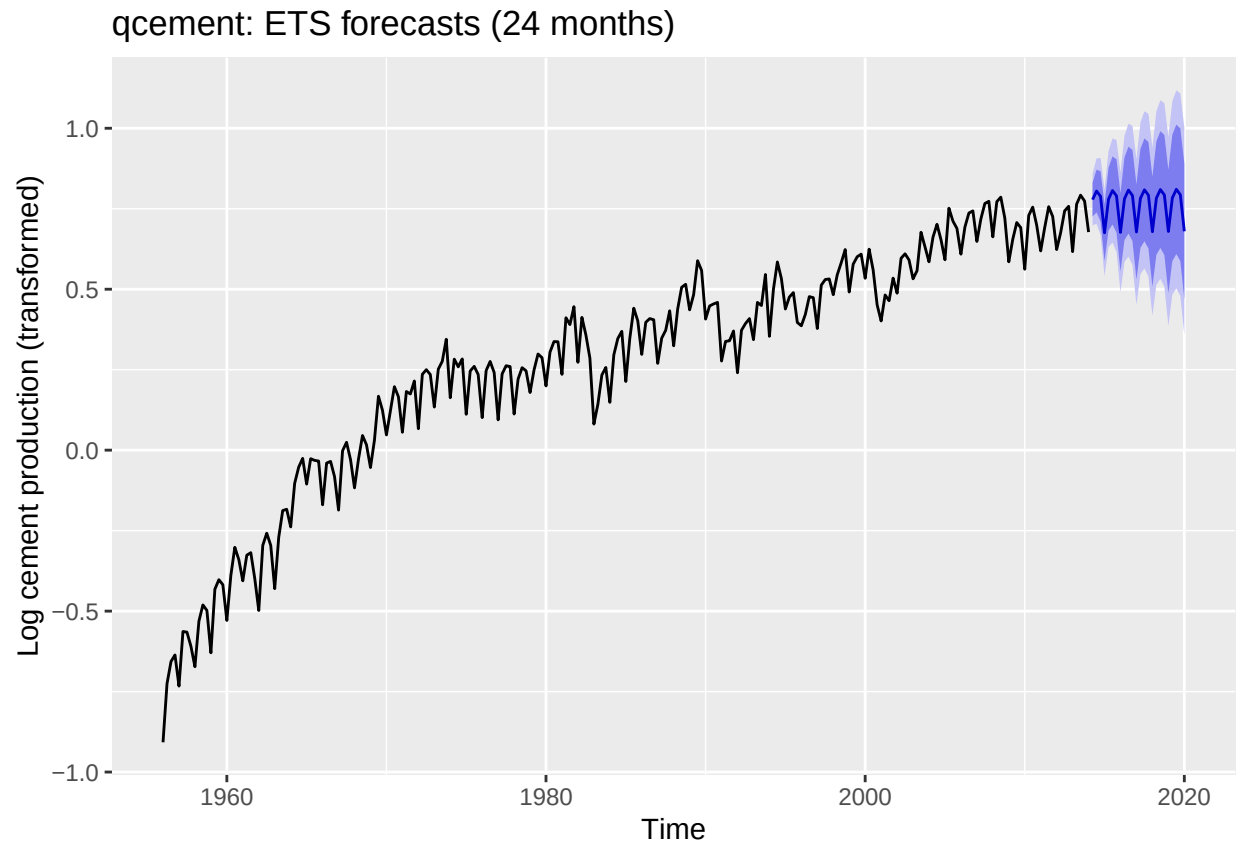
```

## Call:
## ets(y = qclog)
##
## Smoothing parameters:
##   alpha = 0.7697
##   beta  = 0.0049
##   gamma = 1e-04
##   phi   = 0.9492
##
## Initial states:
##   l = -0.8472
##   b = 0.0278
##   s = 0.0265 0.0435 0.0171 -0.0871
##
## sigma: 0.0409
##
##      AIC      AICc      BIC
## -208.3435 -207.3526 -173.8332
##
## Training set error measures:
##              ME      RMSE      MAE      MPE      MAPE      MASE
## Training set 0.005461006 0.04013418 0.03209716 -16.20319 30.61051 0.5337073
##              ACF1
## Training set -0.0183556

fc_ets <- forecast(fit_ets, h = 24)

autoplot(fc_ets) +
  ggtitle("qcement: ETS forecasts (24 months)") +
  ylab("Log cement production (transformed)")

```



The ETS forecast is very similar to the ARIMA forecast. The interval seems to be the same, but the trend stops increasing, making me favor the ARIMA forecast. It seems unrealistic to assume that the data will continue at a horizontal rate rather than the rate we had observed historically.

8.18

a)

```
library(Quandl)
```

Quandl Error

```
## Loading required package: xts
```

```
## Loading required package: zoo
```

```
##
```

```
## Attaching package: 'zoo'
```

```
## The following object is masked from 'package:tsibble':
```

```
##
```

```
## index
```

```
## The following objects are masked from 'package:base':
##
##      as.Date, as.Date.numeric

##
## ##### Warning from 'xts' package #####
## #
## # The dplyr lag() function breaks how base R's lag() function is supposed to #
## # work, which breaks lag(my_xts). Calls to lag(my_xts) that you type or #
## # source() into this session won't work correctly. #
## #
## # Use stats::lag() to make sure you're not using dplyr::lag(), or you can add #
## # conflictRules('dplyr', exclude = 'lag') to your .Rprofile to stop #
## # dplyr from breaking base R's lag() function. #
## #
## # Code in packages is not affected. It's protected by R's namespace mechanism #
## # Set 'options(xts.warn_dplyr_breaks_lag = FALSE)' to suppress this warning. #
## #
## #####

##
## Attaching package: 'xts'

## The following objects are masked from 'package:dplyr':
##
##      first, last
```

```
y <- Quandl("QUANTCHA/VOL", api_key="VS9q-JDA9qo4DgRTrik4", type="ts")
```

```
## No encoding supplied: defaulting to UTF-8.
```

```
## Error: <html style="height:100%"><head><META NAME="ROBOTS" CONTENT="NOINDEX, NOFOLLOW"><meta name="f
```

Quandl is not working correctly and thinks I am trying to scrape data, so I'm going to use a built in dataset for 8.18. I hope that's ok.

```
library(fpp2)
y <- uschange[, "Consumption"]

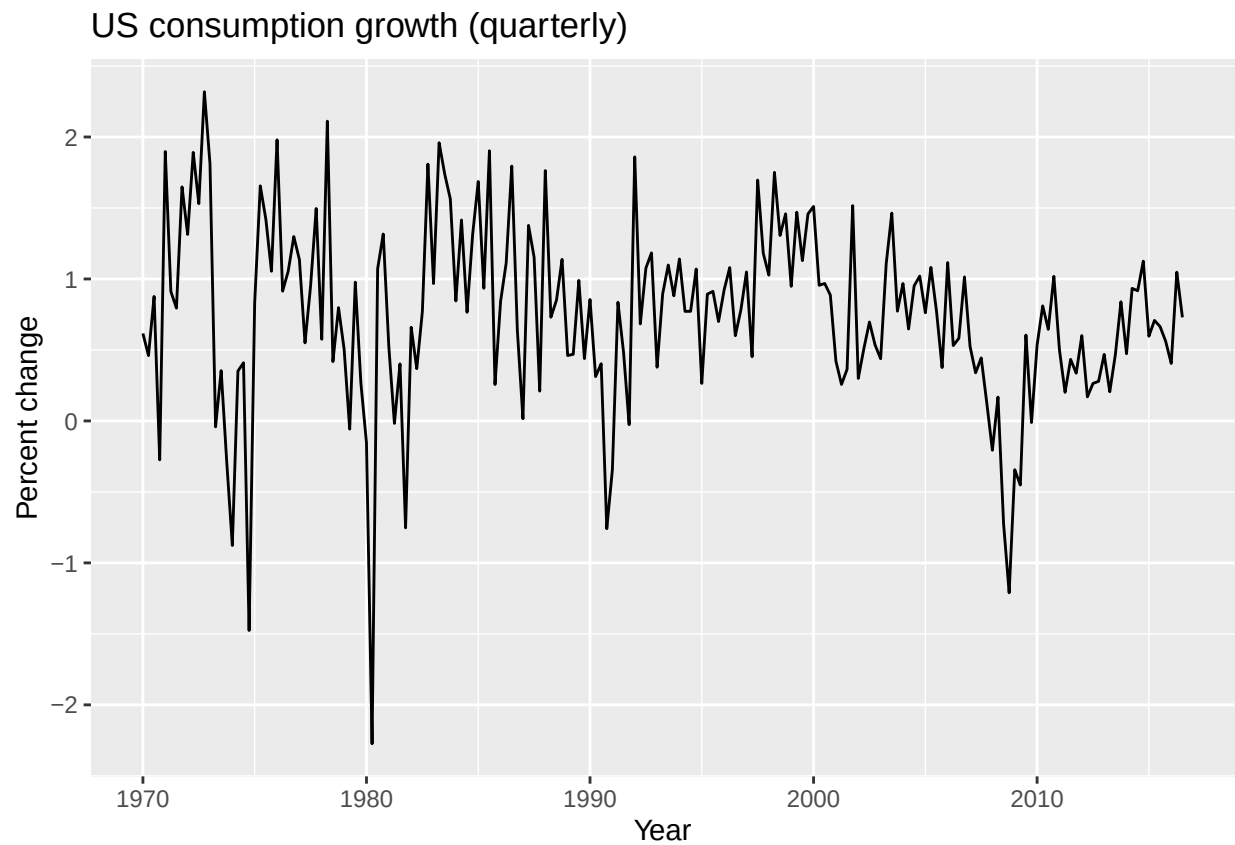
head(y)
```

Data

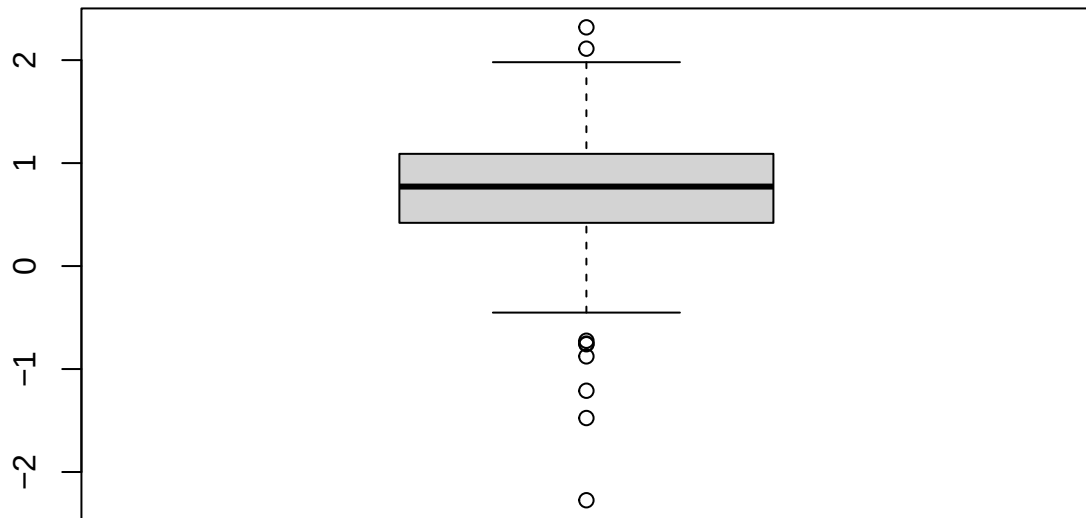
```
##           Qtr1           Qtr2           Qtr3           Qtr4
## 1970  0.6159862  0.4603757  0.8767914 -0.2742451
## 1971  1.8973708  0.9119929
```

b)

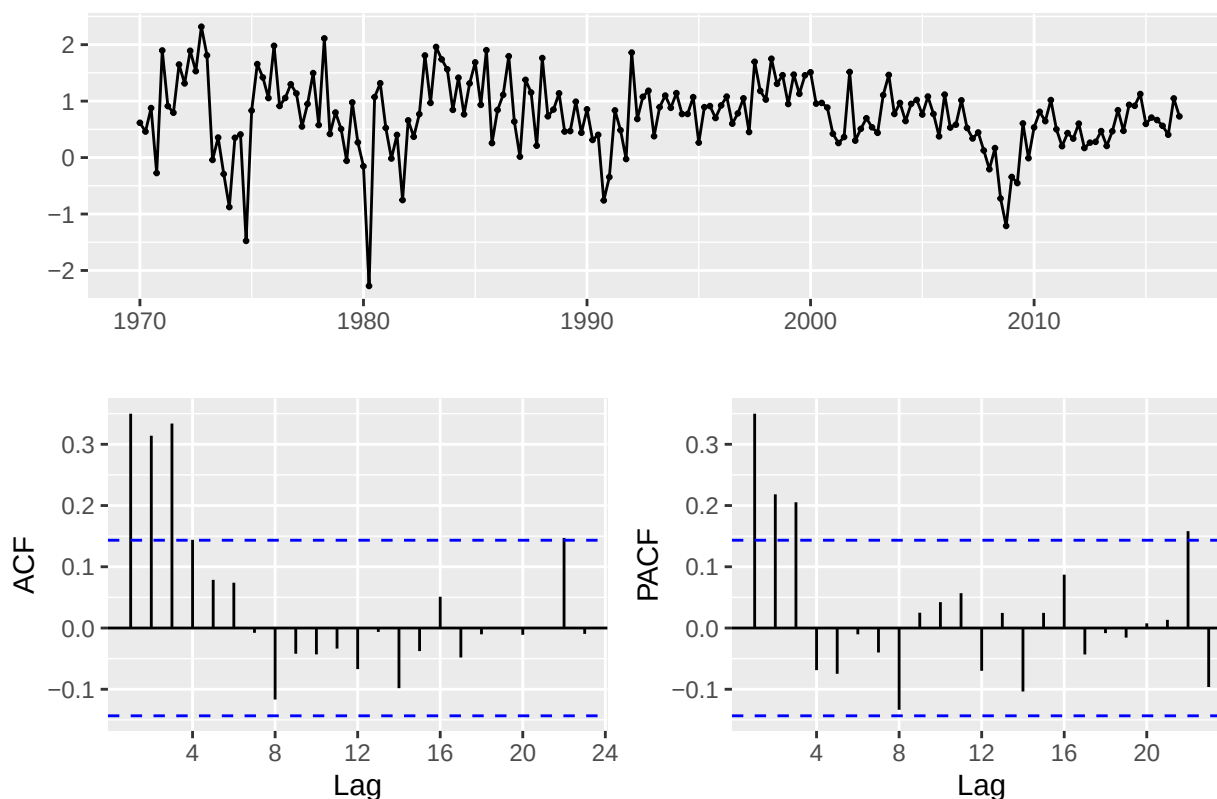

```
autoplot(y) +  
  ggtitle("US consumption growth (quarterly)") +  
  xlab("Year") + ylab("Percent change")
```



```
boxplot(y)
```



```
ggtsdisplay(y)
```



The data has a slight downwards trend and some seasonality but doesn't seem to have many long term cycles. Based on the ACF and PACF spikes, I'd estimate an ARIMA model of ARIMA(3,0,3).

c)

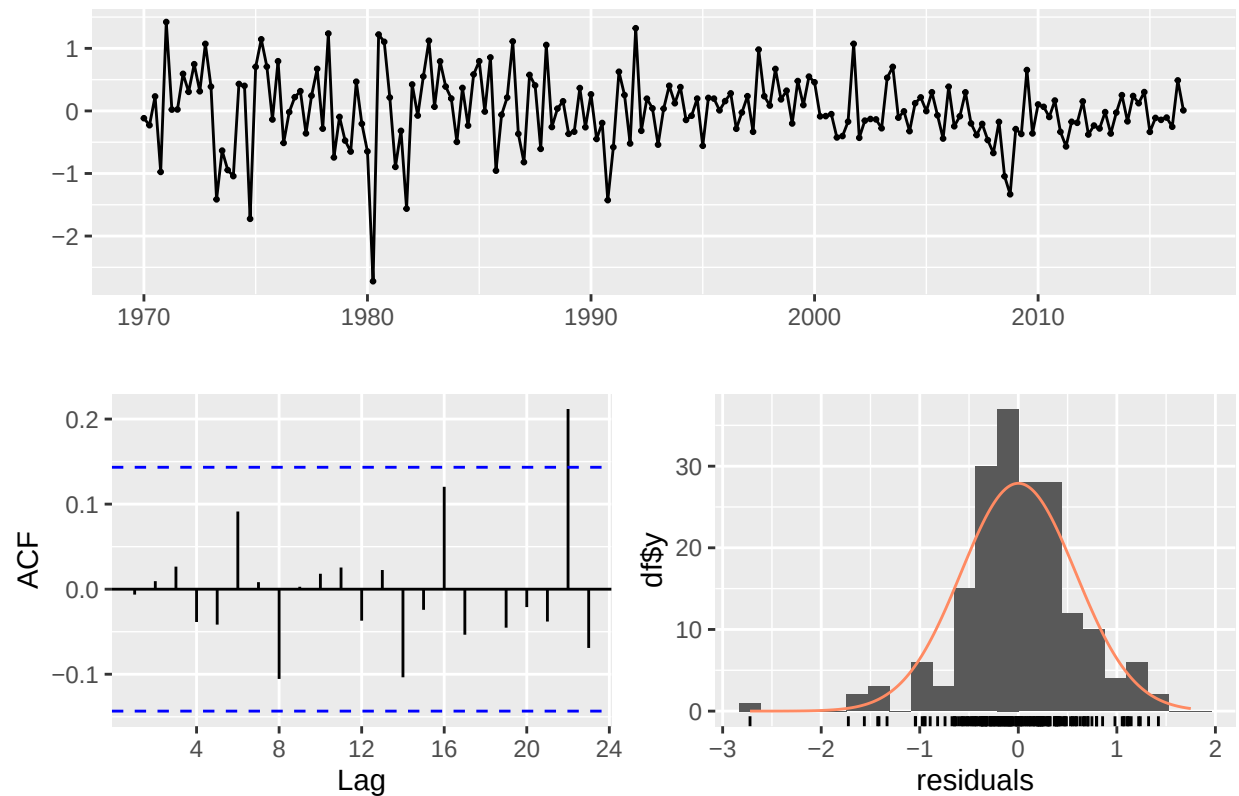
```
fit_arima <- Arima(y, order = c(3, 0, 3))
```

```
summary(fit_arima)
```

```
## Series: y
## ARIMA(3,0,3) with non-zero mean
##
## Coefficients:
##          ar1          ar2          ar3          ma1          ma2          ma3          mean
##          0.5145    0.4530   -0.3675   -0.2769   -0.3564    0.4576    0.7462
## s.e.    0.3272    0.2457    0.2533    0.3111    0.2241    0.1678    0.0871
##
## sigma^2 = 0.3514:  log likelihood = -164.2
## AIC=344.4   AICc=345.21   BIC=370.25
##
## Training set error measures:
##              ME          RMSE          MAE          MPE          MAPE          MASE
## Training set 0.0007268759 0.5815776 0.4296033 54.588 169.4673 0.6731007
##              ACF1
## Training set -0.006358648
```

```
checkresiduals(fit_arima)
```

Residuals from ARIMA(3,0,3) with non-zero mean



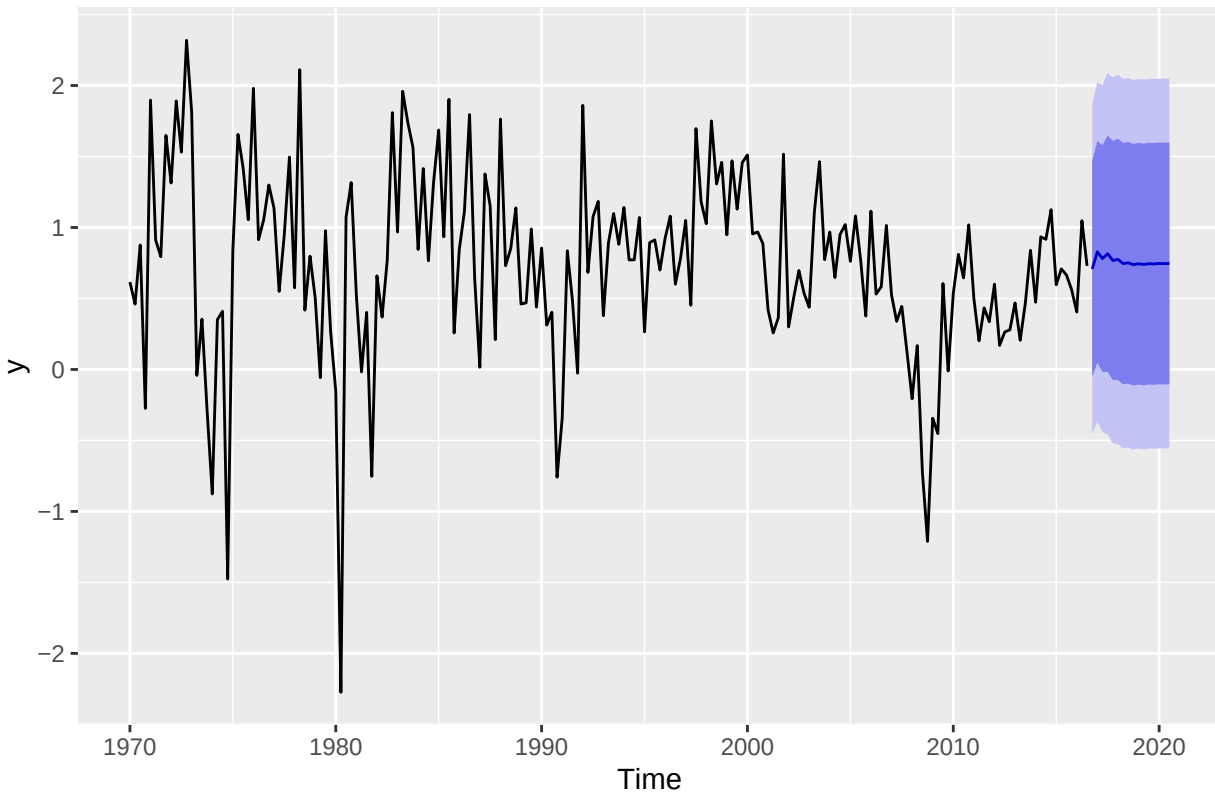
```
##  
##  Ljung-Box test  
##  
## data:  Residuals from ARIMA(3,0,3) with non-zero mean  
## Q* = 4.6243, df = 3, p-value = 0.2015  
##  
## Model df: 6.    Total lags used: 9
```

Looking at the Ljung-Box test p value, which is over 5%, we can't rule out the null hypothesis and declare autocorrelation. The residuals confirm this. They are normally distributed, seem to follow a white noise pattern in the time series plot, and don't have any patterns in the ACF.

d)

```
h <- 4 * frequency(y)  
fc_arima <- forecast(fit_arima, h = h)  
autoplot(fc_arima)
```

Forecasts from ARIMA(3,0,3) with non-zero mean



The forecast is not very useful. It is almost completely devoid of trend or seasonality.

e)

```
fit_ets <- ets(y)
summary(fit_ets)
```

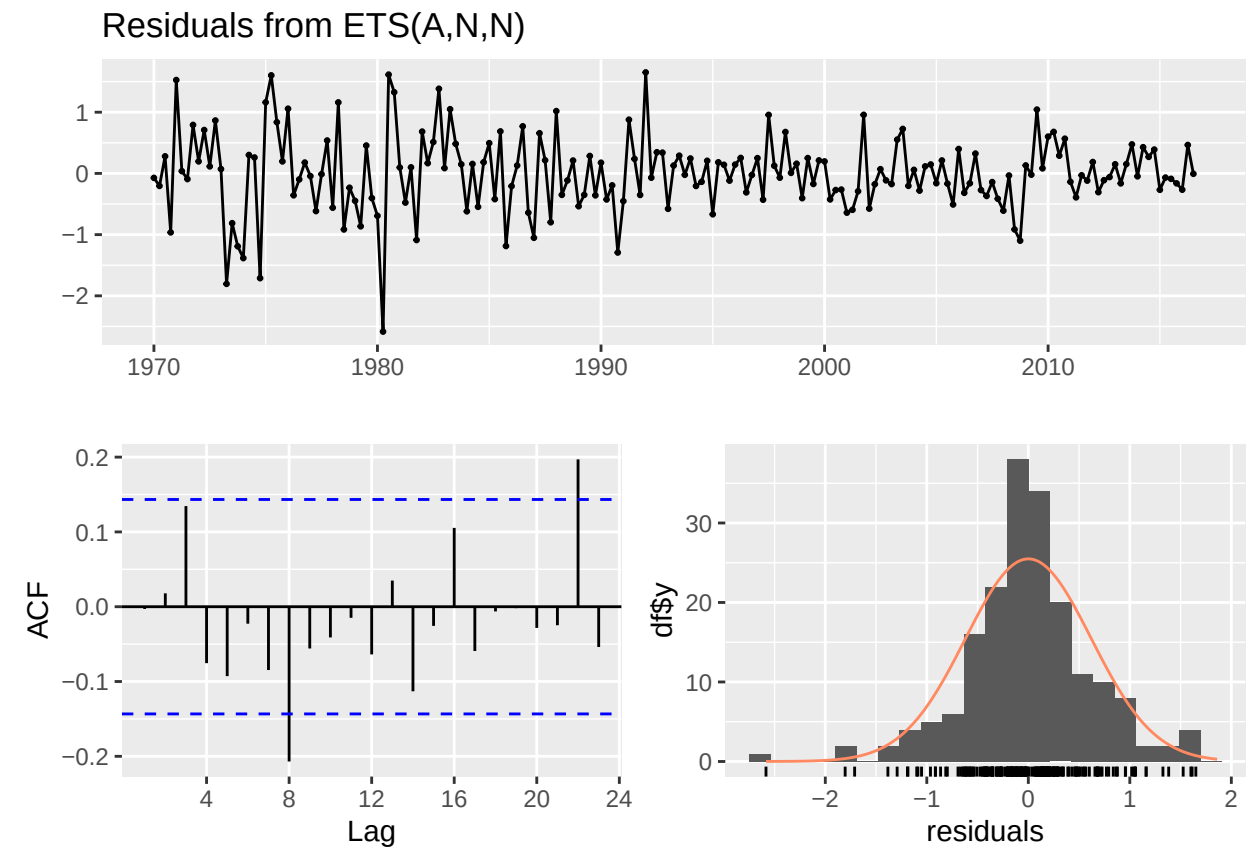
```
## ETS(A,N,N)
##
## Call:
## ets(y = y)
##
## Smoothing parameters:
##   alpha = 0.3298
##
## Initial states:
##   l = 0.6889
##
## sigma: 0.6215
##
##      AIC      AICc      BIC
## 804.3353 804.4665 814.0287
##
## Training set error measures:
##                ME      RMSE      MAE      MPE      MAPE      MASE
```

```
## Training set 0.0007327055 0.6181848 0.4517208 14.72506 157.9753 0.7077542
## ACF1
## Training set -0.003046691
```

Based on this output, we need an ETS(A,N,N) with $\alpha = 0.3298$

f)

```
checkresiduals(fit_ets)
```



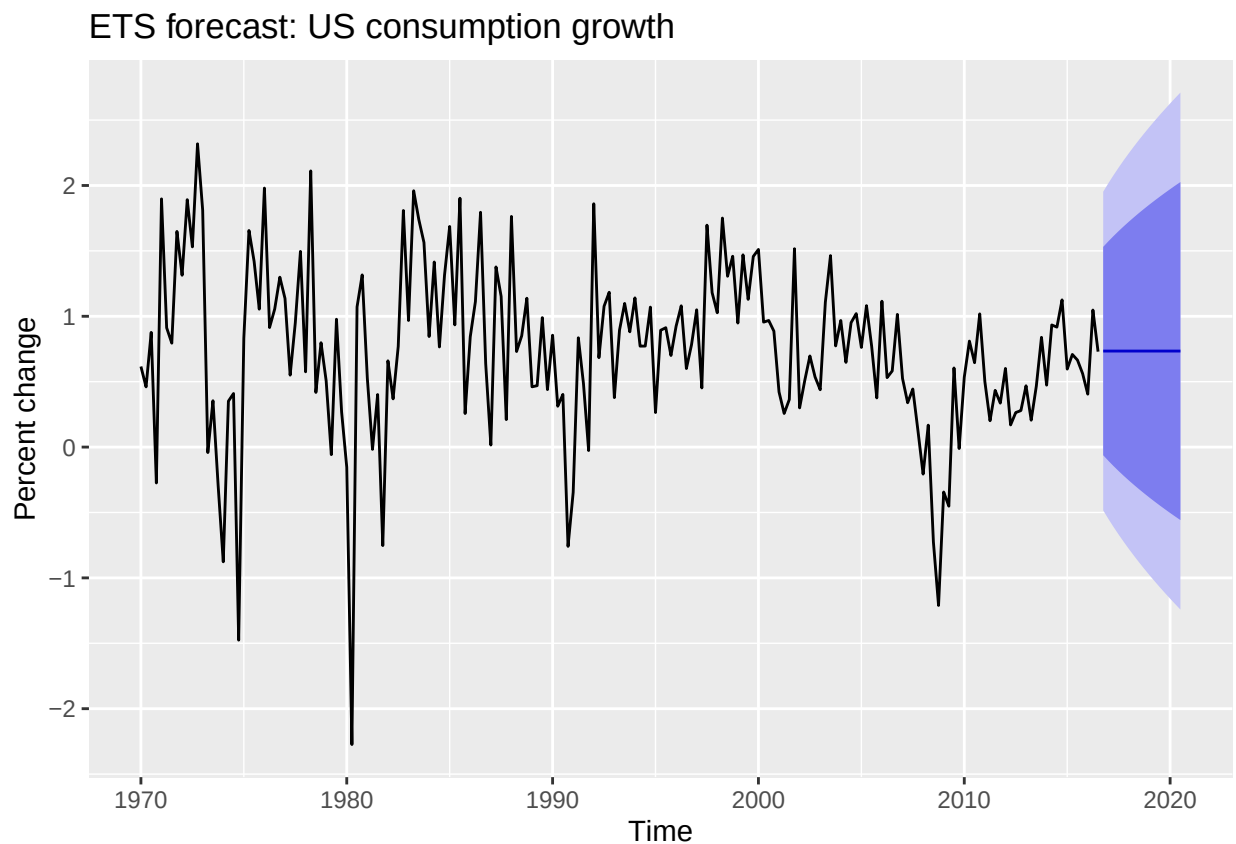
```
##
## Ljung-Box test
##
## data: Residuals from ETS(A,N,N)
## Q* = 16.27, df = 8, p-value = 0.03867
##
## Model df: 0. Total lags used: 8
```

In the ets summary, we see that the p value for the Ljung-Box test is less than 0.05, so we can reject the null hypothesis and claim that the model doesn't fully account for the autocorrelations. The residual time series is pretty random, but the histogram is not very bellcurve shaped and the ACF plot has a few early spikes.

g)

```
fc_ets <- forecast(fit_ets, h = h)

autoplot(fc_ets) +
  ggtitle("ETS forecast: US consumption growth") +
  ylab("Percent change")
```



The ETS forecast is completely horizontal and unconvincing. The CI bounds increase, but the actual forecast doesn't tell us anything.

h)

```
rmse_arima <- sqrt(mean(residuals(fit_arima)^2, na.rm = TRUE))
rmse_ets <- sqrt(mean(residuals(fit_ets)^2, na.rm = TRUE))
rmse_arima
```

```
## [1] 0.5815776
```

```
rmse_ets
```

```
## [1] 0.6181848
```

Based on the RMSE values, we prefer the ARIMA model. It better accounts for the autocorrelations than the ETS (not by much).

12.4 Method on New Data

Data

```
btc <- read.csv("bitcoin.csv", sep = ";")

btc$Date <- as.Date(sub("T.*", "", btc$timeOpen))
btc <- btc[order(btc$Date), ]

y <- ts(btc$close, frequency = 7)

h      <- 30
train  <- head(y, length(y) - h)
test   <- tail(y, h)

head(y)

## Time Series:
## Start = c(1, 1)
## End = c(1, 6)
## Frequency = 7
## [1] 67399.83 68418.79 68362.73 69001.71 67367.85 67361.40
```

Code

```
ETS <- forecast(ets(train), h = h)

ARIMA <- forecast(
  auto.arima(train, lambda = 0, biasadj = TRUE),
  h = h
)

STL <- stlf(
  train,
  lambda = 0,
  h = h,
  biasadj = TRUE
)

NNAR <- forecast(nnetar(train), h = h)

TBATS <- forecast(tbats(train, biasadj = TRUE), h = h)

Combination <- (ETS[["mean"]] +
  ARIMA[["mean"]] +
  STL[["mean"]] +
  NNAR[["mean"]])
```

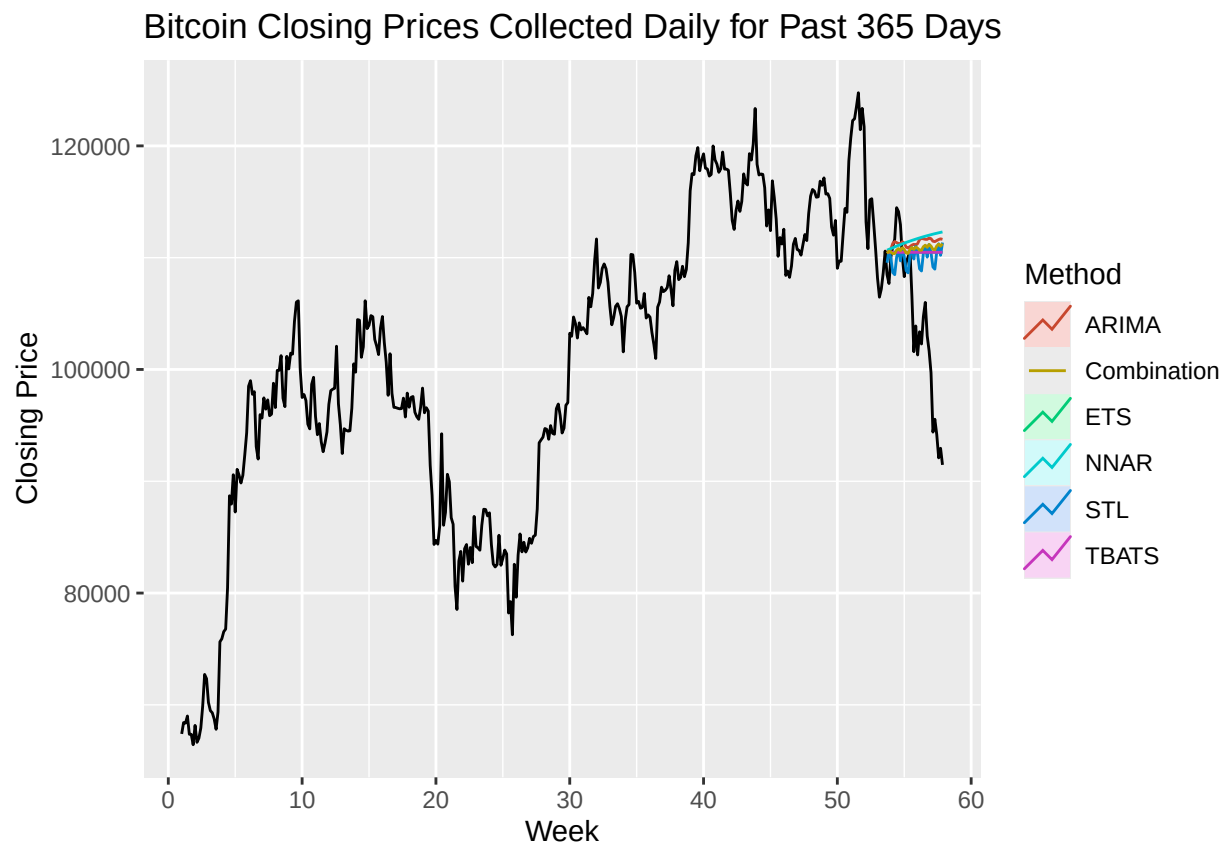


```

TBATS[["mean"]]) / 5

autoplot(y) +
  autolayer(ETS, series = "ETS", PI = FALSE) +
  autolayer(ARIMA, series = "ARIMA", PI = FALSE) +
  autolayer(STL, series = "STL", PI = FALSE) +
  autolayer(NNAR, series = "NNAR", PI = FALSE) +
  autolayer(TBATS, series = "TBATS", PI = FALSE) +
  autolayer(Combination, series = "Combination") +
  xlab("Week") + ylab("Closing Price") +
  ggtitle("Bitcoin Closing Prices Collected Daily for Past 365 Days") +
  guides(colour = guide_legend(title = "Method"))

```



```

rmse_tab <- c(
  ETS      = accuracy(ETS, y)[ "Test set", "RMSE"],
  ARIMA    = accuracy(ARIMA, y)[ "Test set", "RMSE"],
  STL_ETS  = accuracy(STL, y)[ "Test set", "RMSE"],
  NNAR     = accuracy(NNAR, y)[ "Test set", "RMSE"],
  TBATS    = accuracy(TBATS, y)[ "Test set", "RMSE"],
  Combination = accuracy(Combination, y)[ "Test set", "RMSE"]
)
rmse_tab

```

##	ETS	ARIMA	STL_ETS	NNAR	TBATS	Combination
##	9045.046	9756.229	8957.879	10104.433	9042.585	9366.802

Comments

Running the same procedure as 12.4 on Bitcoin data, we see that none of the models accurately predict the recent crypto collapse. I used 11 months as training data and 1 month as test data. The past month happens to have been very negative for bitcoin, but every model predicted stability or an increase in closing price. The closest was the STL plot, which had bitcoin declining more than the other models.